FH JOANNEUM
Applied Computer Sciences

MASTER'S THESIS

A Specification-Based Intrusion Detection System for CANopen Networks on Embedded Hardware

Submitted at

FH JOANNEUM
Master's Degree Programme
"IT and Mobile Security"

Author

Sandjar Berdiev
51812418
Cohort IMS 2024

Supervisor

DI (FH) Florian Temel, MSc MSc

Kapfenberg, July 14, 2026 Signature

Declaration of Honour:

I hereby declare under oath,

- that I have independently prepared this Bachelor/Master thesis and have performed all associated tasks myself, using no other sources or aids than those indicated.
- that in preparing the thesis I have adhered to the guidelines of FH JOANNEUM for ensuring good scientific practice and for avoiding misconduct during the preparation of this work,
- that I have properly cited all formulations and concepts taken over from printed, unprinted works as well as from the Internet in wording or in the essential content in accordance with the rules of Good Scientific Practice (guideline GSP) and have marked them by precise references,
- that I have declared in the method presentation or an index all aids used (artificial intelligence assistance systems such as chatbots [e.g., ChatGPT], translation applications [e.g., DeepL], paraphrasing applications [e.g., Quill bot], image generator applications [e.g., Dall-E], or programming applications [e.g., Github Copilot]) and indicated their usage at the corresponding text passages.
- that this original thesis, in its current form, has not been submitted to any other academic institution for the purpose of obtaining an academic degree.

I have been informed that my work may be checked for plagiarism and for third-party authorship of human (ghostwriting) or technical origin (artificial intelligence assistance systems).

I am aware that a false statement may result in legal consequences such as a negative assessment of my work, the subsequent revocation of any obtained degree, and legal prosecution.

(Signature) (Place, Date)

Kurzfassung:

CANopen ist ein höheres Protokoll, das auf dem Controller Area Network (CAN) aufbaut. Es wird in der industriellen Automatisierung weit verbreitet eingesetzt, besitzt aber keine Sicherheit auf Protokollebene. Jeder Knoten am Bus gilt als vertrauenswürdig, sodass ein einzelner gefälschter Frame als echt akzeptiert wird. Diese Arbeit entwirft, baut und bewertet ein spezifikationsbasiertes Angriffserkennungssystem für CANopen, das auf kostengünstigen Mikrocontrollern läuft.

Das System wird auf einem Red Team und Blue Team Prüfstand auf ESP32 Hardware bewertet. Ein Red Node greift den Bus über sieben CANopen Services an, ein Victim Node ist das Ziel, und ein Blue Node betreibt den Detektor. Die Angriffskommandos laufen über einen separaten Kanal außerhalb des Busses, sodass der Detektor nur auf die Frames reagiert, die die Angriffe erzeugen.

Zwei Forschungsfragen leiten die Arbeit. Die erste fragt, ob Angriffe über die sieben untersuchten Kategorien Signaturen hinterlassen, die sich vom normalen Verkehr unterscheiden lassen. Die zweite fragt, welche Erkennungsregeln aus den Spezifikationen CiA 301 und CiA 305 abgeleitet werden können. Da die Regeln aus der Spezifikation stammen und nicht aus dem Angriffsverkehr, kann der Detektor auch bisher ungesehene Varianten melden.

Die Ergebnisse stützen beide Fragen. Jede Kategorie erzeugte eine unterscheidbare und wiederholbare Signatur, und der Detektor löste jedes Mal den richtigen Alarm aus. Das zentrale Ergebnis vergleicht zwei Detektormodi. Ein Transportschichtmodus bewertet einen Frame nach seinem Identifier und seinem Zeitverhalten. Ein CANopen bewusster Modus bewertet zusätzlich seine Bedeutung. Einige Angriffe wurden nur im CANopen bewussten Modus erkannt, weil sie einen gültigen Identifier wiederverwenden und einer Transportschichtregel nichts Auffälliges bieten. Ein CAN Detektor und ein CANopen Detektor sind sich ergänzende Schichten. Dies zu zeigen ist der zentrale Beitrag dieser Arbeit.

Schlüsselwörter: CANopen | Angriffserkennung | spezifikationsbasierte Erkennung | CAN-Bus-Sicherheit | industrielle Steuerungssysteme | ESP32

Abstract:

CANopen is a higher layer protocol built on the Controller Area Network (CAN), used widely in industrial automation, yet it has no security at the protocol level. Every node on the bus is trusted, so a single forged frame is accepted as genuine. This thesis designs, builds, and evaluates a specification based intrusion detection system for CANopen that runs on low cost microcontrollers.

The system is evaluated on a red team and blue team testbed built on ESP32 hardware. A red node attacks the bus across seven CANopen service categories, a victim node serves as the target, and a blue node runs the detector. Attack commands travel over a separate channel, off the bus, so the detector reacts only to the frames they produce.

Two research questions guide the work. The first asks whether attacks across the seven studied categories leave signatures that can be told apart from normal traffic. The second asks which detection rules can be derived from the CiA 301 and CiA 305 specifications. Because the rules come from the specification, not the attack traffic, the detector can also flag variants it has never seen.

The results support both questions. Every category produced a distinguishable and repeatable signature, and the detector raised the correct alert each time. The central result compares two detector modes. A transport layer mode judges a frame by its identifier and timing. A CANopen aware mode also judges its meaning. Some attacks were caught only in the CANopen aware mode, because they reuse a valid identifier and leave nothing unusual for a transport layer rule to see. A CAN detector and a CANopen detector are complementary layers. Showing this is the main contribution of this thesis.

Keywords: CANopen | intrusion detection | specification based detection | CAN bus security | industrial control systems | ESP32

Contents

1. Introduction

This thesis studies the security of CANopen networks on embedded hardware. CANopen is a higher layer protocol on top of the Controller Area Network. It is used widely in industrial automation, but it has no security at the protocol level. The thesis builds a security testbed on low cost microcontrollers and uses it to study both sides of this problem. One part is a systematic attack tool that targets the CANopen services. The other part is a specification based intrusion detection system that checks live traffic against the protocol specification. The work examines whether the attacks leave signatures that can be told apart, and whether a specification based detector can run passively on microcontroller hardware. The aim is to give a concrete and tested method to protect CANopen systems, which the literature so far does not provide.

1.1. Background

The Controller Area Network (CAN) is a serial bus protocol. Bosch developed it in the early 1980s. It connects sensors, actuators, and controllers in real time. Today CAN is the de facto standard in many networked embedded systems. It is found in cars, boats and spacecraft. It is also used widely in industrial automation, production machinery, medical devices, and building automation (Johansson, Törngren, and Nielsen, 2005).

CAN only defines the lower layers of communication. It defines how bits travel on the wire and how frames are arbitrated. It does not define how applications should use those frames. For this reason higher layer protocols were built on top of CAN. CANopen is one of the most important of these protocols. The standard CiA 301 defines the CANopen application layer and communication profile (CAN in Automation (CiA), 2011). If CAN is the highway, then CANopen adds the traffic rules. It adds rules to the bus so that every device which speaks the CANopen language can understand one another. To do this it provides an object dictionary, network management, and standard message types. These features let devices from different vendors work together.

CANopen is built for modularity. This is its central design idea. A system is not one large device. Instead it is many small modules joined on one bus. Each module has its own role. One module could be a motor drive. Another could be a sensor. Another could be a controller. The result is a distributed system that can grow over time.

1.2. The CANopen Services and Their Identifiers

The attacks and the detection in this thesis act on the CANopen communication services defined in CiA 301 (CAN in Automation (CiA), 2011). Each service uses a CAN identifier, called the Communication Object Identifier (COB-ID). For most services the COB-ID is a fixed function code plus the node ID, so the identifier changes with the node. A few services are global and use a fixed identifier that does not depend on the node ID, because the node is instead addressed in the data field. Table 1.1 lists both the base function code and the resulting COB-ID with node ID `0x01`, which is the victim used in this work. The services marked as fixed keep the same identifier for every node.

Service	Function	Base	COB-ID
NMT	Controls node state (operational, stopped, etc.)	0x000 (fixed)	0x000
SYNC	Network wide synchronisation message	0x080 (fixed)	0x080
EMCY	Reports a fault in a node	0x080	0x081
TPDO1	Transmit process data 1 (first cyclic)	0x180	0x181
TPDO2	Transmit process data 2 (second cyclic)	0x280	0x281
SDO (req)	Object dictionary access, client to server	0x600	0x601
SDO (res)	Object dictionary access, server to client	0x580	0x581
Heartbeat	Periodic alive signal and node state	0x700	0x701
LSS (req)	Sets node ID and bit rate, master to slave	0x7E5 (fixed)	0x7E5
LSS (res)	Layer setting service, slave to master	0x7E4 (fixed)	0x7E4

Table 1.1.: CANopen communication services.

The services divide into two groups by their role. One group manages the network and the state of each node. The other group carries the actual application data. The paragraphs below describe each service in turn.

1.2.1. Object Dictionary and CAN Frame

Two terms are used throughout this thesis and are defined here before they appear in the service descriptions.

The object dictionary is the list of all data a CANopen device exposes. It is a table inside the device, in which every value has its own row. A row holds a process value, for example a temperature value, or a setting. Each row is found by a 16-bit index, and a row with several parts uses an 8-bit sub-index for each part. CiA 301 fixes the layout of this table (CAN in Automation (CiA), 2011). The object dictionary is the source of the data, and the communication services are the transport that moves it.

A CAN frame is a single message on the bus. It is the smallest unit the bus sends. One frame has an identifier, the COB-ID, which states the type of the message and the node it concerns, and a data field of up to eight bytes. Every CANopen service, an NMT command, a heartbeat, or a PDO, travels as one or more of these frames. To map data into one frame means to pack values from the object dictionary into the eight data bytes of a single CAN message.

1.2.2. Master / Slave Roles

In CANopen the roles of master and slave describe control. CiA 301 does not define one single master for the whole network. It defines the master as a role for one functionality, where exactly one device acts as the master for that functionality and all other devices are slaves (CAN in Automation (CiA), 2011). A device can be the master for one functionality and a slave for another.

The NMT master is the master for the network management functionality. It is the node that controls the state of the slaves. It sends the NMT commands that move slaves between the pre-operational, operational, and stopped states. CiA 301 requires that one device in the network fulfils the role of the NMT master (CAN in Automation (CiA), 2011). A slave is a node controlled by the master. For example a sensor or a drive, that receives these commands and follows them.

Other control roles are separate from the NMT master. The node that configures the node identifier and bit rate is the LSS master, which is defined in CiA 305 (CAN in Automation

(CiA), 2013). One physical device often fills several of these roles at once, which is why it is loosely called the master.

1.2.3. Network Management Service

The Network Management (NMT) service controls the state of each node. CiA 301 defines a node state machine with the states pre-operational, operational, and stopped. A node enters the pre-operational state after the boot-up message. In this state it can use SDOs but it cannot send or receive PDOs. In the operational state the node can use all services, including PDOs. In the stopped state the node only processes NMT and heartbeat messages. The NMT service sends short commands on COB-ID `0x000` to move a node between these states. This identifier is fixed and the target node is named in the data field (CAN in Automation (CiA), 2011).

The NMT command is a single CAN frame with COB-ID `0x000` and two data bytes. The first byte is the command specifier, which names the target state. The second byte is the node ID of the device that must perform the command. If the second byte is `0x00`, then every node on the bus performs the command at once (CAN in Automation (CiA), 2011). For node ID `0x01` the commands are listed in Table 1.2

Command	**Resulting state**	**Data bytes**
Start node	Operational	`0x01 0x01`
Stop node	Stopped	`0x02 0x01`
Enter pre-operational	Pre-operational	`0x80 0x01`
Reset node	Initialisation	`0x81 0x01`
Reset communication	Initialisation	`0x82 0x01`

Table 1.2.: NMT command frames to control node `0x01`.

To start node `0x01`, the NMT master sends `0x01 0x01`. To stop the same node, it sends `0x02 0x01`. To put it back into the pre-operational state, it sends `0x80 0x01`. A broadcast start to all nodes is `0x01 0x00`.

The NMT command should be sent by the NMT master. CiA 301 places the network control role with a single active NMT master, and that master is the node allowed to issue these state transitions (CAN in Automation (CiA), 2011). A slave is a managed node that receives the command and changes state. It should not send NMT commands in normal operation.

However in raw CAN terms, the protocol cannot stop other nodes from sending this frame. The NMT command is just a CAN frame with COB-ID `0x000` and two data bytes, and CAN has no source address and no authentication. Any node on the bus can place this frame on the wire, and every node will accept it. This open control path is one of the weaknesses that the attacks in this thesis target.

1.2.4. Process Data Objects

Process Data Objects (PDOs) carry fast cyclic process data, such as sensor values or motor commands (CAN in Automation (CiA), 2011). The PDO is named from the point of view of each device. A node that sends the data uses a transmit PDO (TPDO) and is the PDO producer. A node that receives the same data uses a receive PDO (RPDO) and is the PDO consumer. So one message on the bus is a TPDO for the sender and an RPDO for the receiver (CAN in Automation (CiA), 2011).

A CANopen device may define up to 512 PDOs in each direction through the object dictionary (CAN in Automation (CiA), 2011). When a node must transfer large or arbitrary data

rather than small cyclic values, it uses an SDO transfer instead of a PDO, because SDOs are confirmed and can carry large blocks.

So one message on the bus is a TPDO for the sender and an RPDO for the receiver. The producer and consumer roles describe data flow and are separate from the master and slave roles, so a single slave can be the producer of its own process data while still being a slave for network control.

One producer can have many consumers, because CAN is a broadcast bus. One node produces a TPDO and several nodes consume it as their RPDO. This same model applies to the heartbeat and emergency services, where one node produces the message and other nodes consume it. The model has a security consequence that this thesis relies on. A consumer accepts a frame by its COB-ID alone. Any node on the bus can produce a frame that every consumer treats as genuine. This is what makes process data, heartbeat, and emergency message spoofing possible.

1.2.5. Service Data Objects

Service Data Objects (SDOs) give confirmed access to the object dictionary of a node (CAN in Automation (CiA), 2011). An SDO transfer is a request and response pair. The node that starts the transfer is the SDO client, and the node whose object dictionary is accessed is the SDO server. The client sends a request on COB-ID `0x600` plus the node ID, and the server answers on COB-ID `0x580` plus the node ID. Every transfer is confirmed, and a large object is split across several frames, so an SDO has more overhead than a PDO. SDOs are therefore used for configuration and for infrequent or large data transfers, not for fast cyclic process data.

1.2.6. Synchronisation Service

The Synchronization (SYNC) service provides a common time base for the network. One node produces a periodic SYNC message on the fixed COB-ID `0x080`. Other nodes use this message to sample their inputs or to apply their outputs at the same moment (CAN in Automation (CiA), 2011). The SYNC message lets synchronous PDOs act together across the bus.

1.2.7. Heartbeat Service

The Heartbeat service lets a node report that it is alive. The node sends a periodic message on COB-ID `0x700` plus the node ID, and this message also reports the current NMT state of the node (CAN in Automation (CiA), 2011). The producer period is set in object `0x1017`, and a consumer sets its timeout in object `0x1016`. Other nodes watch this signal, and if no heartbeat arrives within the expected time, the consumer treats the node as failed. The heartbeat gives passive monitoring of node health without polling it.

1.2.8. Emergency Service

The Emergency (EMCY) service reports a fault inside a node. When a device detects an internal error it sends an emergency message on COB-ID `0x080` plus the node ID, and the message carries an error code that describes the fault (CAN in Automation (CiA), 2011). The EMCY service informs the rest of the network that the node has entered an error condition.

1.2.9. Layer Setting Service

The Layer Setting Services (LSS) configure the node identifier and the bit rate of a device on the bus (CAN in Automation (CiA), 2013). LSS is used before normal operation, often when a

device has no valid node ID yet. The LSS master sends requests on the fixed COB-ID `0x7E5`, and the LSS slave answers on the fixed COB-ID `0x7E4`. Because LSS can change the identity of a node, it is a sensitive configuration service.

1.2.10. Specification as the Basis for Detection

Each of these services follows clear rules. CiA 301 defines the rules for network management, process data, service data, synchronization, heartbeat, and emergency objects (CAN in Automation (CiA), 2011), and CiA 305 defines the rules for the layer setting service (CAN in Automation (CiA), 2013). The rules state which messages are allowed and which identifier each message must use. Correct traffic must follow these rules, and traffic that breaks them is suspicious. This is the specification that a detection system can check live traffic against, and it is the foundation of the approach used in this thesis.

1.3. Problem Statement and Research Gap

CAN bus security has been studied extensively, with most work targeting the automotive domain (Rajapaksha et al., 2023). Researchers have built detectors using deep learning, timing analysis, and physical layer features, and they have built authentication and encryption schemes for CAN (Lotto et al., 2024; Cui et al., 2023; Rasheed et al., 2024). This is a mature and active field.

Research on CANopen is far less developed. Popic and colleagues, in a recent survey of CAN based protocols, note that derivative protocols such as CANopen and J1939 share the same lack of authentication and encryption as raw CAN, and that most defensive work still focuses on the base CAN layer and the automotive case (Popic et al., 2025). The industrial higher layer protocols receive far less attention. The closest safety oriented work for CANopen targets the EtherCAT variant and focuses on safe communication rather than on intrusion detection (Zhengyu et al., 2019).

This creates a clear gap. There is very little implementation focused work on CANopen intrusion detection, and even less on microcontroller class hardware. This matters because CANopen modules are usually small embedded devices. The defense must fit the same tight resources as the device it protects. Recent CAN research has begun to move detection onto small embedded targets (Im and S. Lee, 2024; J. Lee et al., 2025), but this work targets automotive CAN and not the CANopen application layer.

These existing detectors raise a fair question. If a CAN intrusion detection system already exists, why build one for CANopen. A CAN detector inspects how frames are sent. It works on the identifier, the timing, the bus load, and the physical signal. It detects any CANopen attack that disturbs these properties, such as an injected process data frame at the wrong time or a flood that raises the bus load. For a large share of attacks this is enough. It is not enough for an attack that is valid at the transport layer. A service data write that rewrites the object dictionary of a node uses the correct identifier, sends a well formed frame, and adds no measurable load, because such transfers are aperiodic by design. To a CAN detector nothing is anomalous, because at the transport layer nothing is anomalous. The attack exists only in the meaning of the frame, where an unauthorized client changes a configuration value it should not touch. A CAN detector cannot see this without modelling the CANopen services, which is to say without becoming a CANopen detector. The two are complementary layers rather than substitutes. A CANopen detector does not replace transport layer monitoring. It covers what transport layer monitoring is blind to by construction.

Figure 1.1 shows why a transport layer detector and a CANopen detector are complementary rather than interchangeable. The lower layer fixes how a frame is sent. The upper layer fixes

what the frame means. An attack that is well formed at the lower layer leaves nothing for a transport layer rule to catch, and only a rule grounded in the CANopen specification can flag it. This is the gap the thesis addresses.

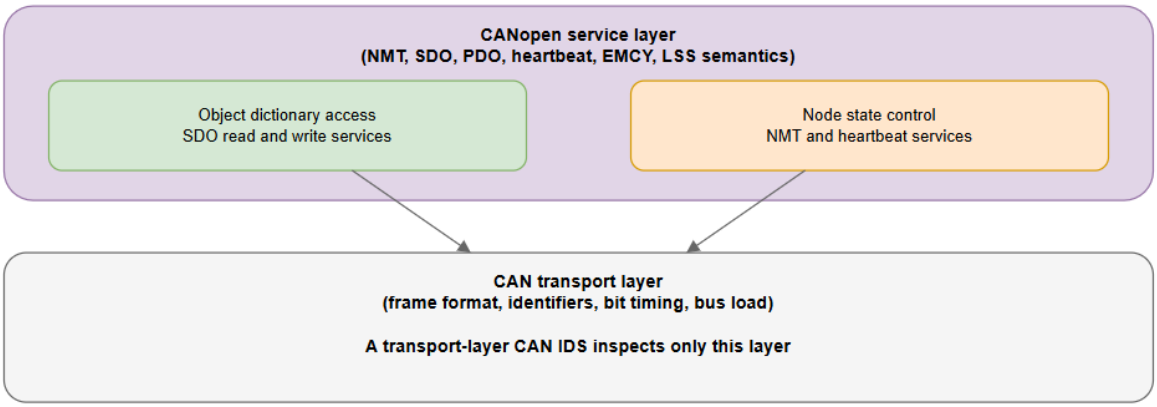

Figure 1.1.: The CANopen detection gap.

The most promising direction for CANopen is specification based detection. Larson, Nilsson, and Jonsson proposed specification based attack detection for in-vehicle networks in 2008 (Larson, Nilsson, and Jonsson, 2008). A protocol has a written specification, correct traffic must follow that specification, and traffic that breaks the specification is suspicious. This idea fits CANopen very well, because CANopen has a detailed public specification in CiA 301, in which the object dictionary, the state machine, and the message rules are all defined.

A 2026 study by Subaşı and colleagues builds a reconfigurable specification driven detector for an electric vehicle powertrain. It enforces timing bounds, identifier whitelists, frame format rules, and bus load thresholds within automotive microcontroller limits, and it reaches detection times below two milliseconds with false positive rates under one percent (Subaşı and Mercimek, 2026). This proves the method can meet real time limits on small devices, but the proof exists for automotive CAN and not yet for CANopen and its specific services.

CANopen is widely used and built for open modular growth, but its services are not secured at the application layer. Specification based detection is a natural fit because CANopen has a rich public specification. No published specification based intrusion detection system has been identified for CANopen. This thesis addresses that gap by designing and evaluating a specification based intrusion detection system for CANopen.

1.4. Research Questions

This thesis is guided by two research questions.

RQ1. Do attacks across major CANopen service categories produce distinguishable and repeatable signatures?

The attack categories are network management state manipulation, service data object abuse, process data object injection, heartbeat manipulation, emergency message injection, layer setting service abuse, and bus level denial of service.

RQ2. Which detection rules for the studied attack categories can be taken from the CANopen specification alone, and which ones also need observed runtime behavior?

The detector is built from the communication rules in CiA 301. RQ2 asks two things. The first is which detection rules for the studied attack categories can be taken from the specification alone. The second is which rules need observed runtime behavior in addition to the specification. Some rules map directly to a clause of CiA 301. Others cannot rely on a clause alone. A process data object injection frame is conformant at the transport layer. Its detection depends on observed transmission behavior, such as the rate at which the frames appear on the bus. RQ2 therefore separates rules that follow from the specification from rules that also need observed runtime behavior. The rules and their clause mapping are presented in Chapter 5, and their behavior against the attacks is reported in Chapter 6.

RQ2 follows from the nature of specification based detection. A specification based detector flags traffic that breaks the written rules of the protocol (Larson, Nilsson, and Jonsson, 2008). Many detection rules correspond directly to a rule that the specification itself defines. Grounding a rule in a clause of CiA 301 makes it traceable and auditable, and it separates this approach from anomaly based methods that learn normal behavior from data. Not every attack can be caught by a clause alone. A frame can be conformant at the transport layer and still be part of an attack, and such a frame is only detected through its observed runtime behavior, such as its timing or its rate on the bus. RQ2 therefore asks which rules follow from the specification alone and which rules also need observed runtime behavior. The limit of the approach is examined in Chapter 6.

1.5. Aim and Scope

The aim of this thesis is to design, build, and evaluate a security testbed for CANopen. The testbed follows a red team and blue team structure and runs on microcontroller hardware.

The red team is a systematic attack tool. It covers the seven studied service categories and produces a systematic and repeatable set of attacks. These attacks supply the data needed to answer RQ1. The blue team is a specification based intrusion detection system. Most of its rules are derived from CiA 301, and each such rule maps to a clause of the specification. Where a clause is not enough, a rule adds an observed runtime property such as message timing or rate. Separating the rules that follow from the specification alone from those that also need observed runtime behavior is what answers RQ2, and the system checks live traffic to detect the attacks from the red team.

The scope is deliberately bounded. The work focuses on the communication profile in CiA 301, with the layer setting service covered by CiA 305, and does not aim to secure every possible CANopen profile. It does not propose new cryptography for CANopen. This thesis instead studies a passive, non intrusive monitoring approach.

1.6. Ethical Considerations

This thesis builds an attack tool, which raises an ethical question that must be addressed openly. The attack tool exists for defensive research only. It runs on a closed and controlled testbed that is not connected to any production system or safety critical machine, and the attacks are never run against real industrial equipment.

The purpose of the attack tool is to generate the data needed to build and test the defense, because a defense can only be trusted if it is tested against real attacks. Building the attacks in a controlled lab is a standard and accepted practice in security research, and the use of dedicated testbeds for this purpose is well documented in the industrial control security literature (Conti,

Donadel, and Turrin, 2021). The knowledge produced here is meant to help operators protect CANopen systems and to support the goals of NIS2 and IEC 62443. It is not meant to enable harm.

2. Motivation

Chapter 1 described what CANopen is, how its services work, and why the protocol has no security at the application layer. This chapter explains why closing this gap matters. The argument has three parts. The first is the convergence of operational and information technology in modern industry. The second is the consequence of an attack, which in industrial machinery can be physical and not only digital. The third is regulation, which has begun to require the protection and monitoring that CANopen systems cannot yet provide.

2.1. Industrial Convergence and Exposure

The open modular design of CANopen, was a strength as long as the bus stayed inside the machine. For many years industrial machines were isolated. The fieldbus was reachable only by a person standing at the device. This is no longer the case. Modern machinery is networked and remotely monitored, which exposes the internal fieldbus to a wider reach (Makrakis et al., 2021). This change is part of the convergence of operational technology and information technology under Industry 4.0. The internal CAN port that CANopen leaves open for modularity is now reachable through this wider system.

Confidentiality often matters less. Authentication and integrity matter more, because they protect safe physical operation (Dzung et al., 2005). A further problem is the long life of industrial systems. They often run for many years without patches, which makes them slow to adopt modern security controls (Nankya, Chataut, and Akl, 2023). A CANopen device installed today may still be running, unchanged, long after the attacks against it are well understood.

2.2. From Data Loss to Physical Harm

In an office system the cost of an attack is usually the loss or theft of data. In an industrial system the cost can be physical. CANopen runs in production machinery and in critical infrastructure, where a manipulated command can move an actuator or disable a protection (Makrakis et al., 2021). The same protocol is also found inside medical and laboratory devices that coordinate sensors and actuators over an internal bus (CAN in Automation (CiA), 2019). An attacker who reaches such a bus does not need to break the device. As Chapter 1 showed, any node on the bus is trusted, so a single injected frame is treated as genuine.

The attacks themselves are not hypothetical at the CAN layer. Raw CAN attacks disrupt and forge. They can deny service by forcing nodes into a bus off state (Murvay and Groza, 2017), and they can inject or spoof frames through spoofing and injection attacks (Bozdal et al., 2020). Because the CAN bus is a broadcast medium, eavesdropping and impersonation are already possible at the raw layer. When the targeted node controls a physical process, these attacks can lead to physical harm and not only to data loss (Makrakis et al., 2021). Proof of concept attacks against medical and industrial devices that use CAN based buses have been reported, and bodies such as ICS-CERT have warned that CAN bus weaknesses can reach healthcare systems (CAN in Automation (CiA), 2019).

CANopen does not add new ways to place frames on the bus. It adds meaning to those frames. An attacker who understands the CANopen services can turn a forged frame into a precise and legitimate looking operation. The attacker can stop a node with an NMT command, silently rewrite a device's configuration through an SDO. The danger of these manipulations is that they need not look like an attack. A configuration change that uses the correct identifier and a well formed frame is a normal CANopen operation in form, and only its meaning is hostile. In an industrial setting this is the worst case. A silent and valid looking change to a node that controls a physical process can move the machine to an unsafe state while every frame on the bus still appears correct. This is also the point at which the manipulation meets regulation, because the law now requires operators not only to prevent such an attack but to detect and report it.

2.3. Regulation Makes the Gap Urgent

Three pieces of European Union law now press on this gap from three different directions. None of them, on its own, tells an operator how to secure CANopen. Together they make the absence of a method a compliance problem and not only a research problem.

The NIS2 directive raises the cybersecurity duties of essential and important entities across the European Union (European Parliament and Council, 2022). It acts on the operators of industrial and critical systems and requires them to manage the risk of their networks. CANopen is one of the protocols those networks run.

The Machinery Regulation (EU) 2023/1230 acts on the machine itself. It replaces the Machinery Directive 2006/42/EC and applies from 20 January 2027 (European Parliament and Council, 2023). For the first time it treats cybersecurity as an essential health and safety requirement, so a machine is no longer safe merely because it is mechanically sound. Connecting another device or gaining remote access must not create a hazardous situation, and safety related control systems and software must be protected against corruption (European Parliament and Council, 2023). This requirement describes, in legal terms, the exact threat studied in this thesis. A hostile module connected to an open CANopen port is the connection of another device that must not lead to a hazardous situation.

The Cyber Resilience Act, Regulation (EU) 2024/2847, acts on the product across its whole lifecycle (European Parliament and Council, 2024). It applies to products with digital elements, which include industrial control systems, and its main obligations apply in full from 11 December 2027 (European Parliament and Council, 2024). Its essential requirements include protection against unauthorised access, the integrity of data and commands against manipulation, and resilience against denial of service (European Parliament and Council, 2024). Each of these maps onto an attack category studied in this thesis. The Machinery Regulation and the Cyber Resilience Act are explicitly linked, and a connected machine placed on the market in late 2027 must satisfy both (European Parliament and Council, 2023).

These three instruments converge on the same industrial machine from the side of the operator and the side of the machine as a safety product. All three now require protection and, in the case of the Cyber Resilience Act, the detection and reporting of incidents. For a protocol such as CANopen, an operator cannot meet these duties without a concrete method that works on the protocol actually in use. As Chapter 1 established, no such published method has been identified. The strength of CANopen has become its weakness, the literature has not closed that gap, and regulation has now made the gap one that operators are required to close. This thesis addresses it.

3. Related Work

This chapter reviews the scientific work that this thesis builds on. It first covers the known attacks on CAN and on CAN based higher layer protocols. It then reviews the main families of intrusion detection, with a focus on the specification based approach and on detection under embedded resource limits.

3.1. Attacks on CAN and CANopen

CAN provides no authentication and no encryption, and this single property is the root cause of most published attacks. Researchers have shown many practical attacks on the bus, and they fall into a few main types.

3.1.1. Injection and spoofing

An attacker sends frames with a forged identifier, and receivers accept them as real. Oladimeji and colleagues showed that a node can spoof a high priority identifier and keep sending it. This blocks trusted frames and can change the behavior of the system (Oladimeji et al., 2023). Public datasets such as CICIoV2024 were built by running spoofing and denial of service attacks on a real car, which shows that realistic evaluation still depends on generating real attack traffic (Neto et al., 2024).

3.1.2. Denial of service

The simplest denial of service attack floods the bus with high priority frames. Because of arbitration these frames win the bus and stop normal traffic, so a single node can block a whole network (Oladimeji et al., 2023).

3.1.3. Bus off attacks

A more advanced attack abuses the error handling system of CAN. Cho and Shin first demonstrated the bus off attack, in which the attacker forces errors on a victim until it enters the bus off state and leaves the bus, even though it was trusted (Cho and Shin, 2016). Later work made the attack stealthier. Iehira and colleagues combined a bus off attack with spoofing, so that the attacker could take over the identity of the victim (Iehira, Inoue, and Ishida, 2018). Bloom showed WeepingCAN, a version that hides the normal signs of the attack (Bloom, 2021). Serag and colleagues found further weaknesses in the error handling rules and used them to map and silence safety critical nodes (Serag et al., 2021).

3.1.4. Attacks on CAN based higher layer protocols

Higher layer protocols inherit the weaknesses above and add their own at the service layer. Murvay and Groza studied the SAE J1939 protocol used in trucks and showed impersonation and denial of service attacks that still follow the standard (Murvay and Groza, 2018). Popic and colleagues surveyed CAN based protocols, listed CANopen, J1939, and NMEA 2000, and noted that all of them lack authentication and encryption (Popic et al., 2025). For CANopen

specifically, the picture is narrower. Surveys of CAN based protocols name CANopen but treat it as one entry in a list and do not study its services in depth (Popic et al., 2025). Dedicated work on attacks against the CANopen service layer, and on detection at that layer, is sparse. The foundational study is by Larson, Nilsson, and Jonsson, who derived security specifications from the CANopen communication profile and object dictionary. They used them to detect attacks at the protocol and node level (Larson, Nilsson, and Jonsson, 2008). Their work shows that the structured services of CANopen can be both a target and a basis for detection. But this work is from 2008 and was used in automotive hardware and did not target the industrial CANopen service layer.

3.2. Families of Intrusion Detection

Because CAN cannot easily be extended with encryption, much defensive work turns to intrusion detection. An intrusion detection system (IDS) monitors traffic and raises an alarm when it observes something that violates expected behavior (Scarfone and Mell, 2007). The standard taxonomy divides detection methods into three families (Axelsson, 2000; Debar, Dacier, and Wespi, 1999).

3.2.1. Signature based detection

This method looks for known attack patterns. It is accurate for known attacks but cannot find new ones, and it needs a signature database that must be kept up to date (Scarfone and Mell, 2007; Khraisat et al., 2019).

3.2.2. Anomaly based detection

This type of detection learns the normal behavior of the system and flags anything that deviates from it. Much recent work on CAN uses machine learning and deep learning, which can find new attacks but often needs large training datasets, heavy computation, and tolerates a high false alarm rate (Rajapaksha et al., 2023).

3.2.3. Specification based detection

This method sits between the two. It does not learn from data and it does not store attack signatures. Instead it uses a model of the correct behavior of the protocol, taken from the standard, and treats any message that breaks the model as an attack (Sekar et al., 2002; Nweke, 2021). This thesis uses this approach, so it is treated in detail in the next section.

3.3. Specification Based Intrusion Detection

Sekar and colleagues first set out the specification based approach in a clear form. They built state machine models of network protocols, added statistics, and detected probing and denial of service attacks in a standard test set with a low false alarm rate (Sekar et al., 2002). The key benefit is that the model is built from the protocol standard and not from attack examples, so the system can detect new and unknown attacks as long as those attacks break the rules. The approach works best when the correct behavior is well defined, which makes industrial and control protocols a good fit because they follow strict standards (Nweke, 2021). It has been applied to control protocols such as DNP3 (Lin et al., 2013) and to hierarchical industrial control systems (Hotellier et al., 2024).

For CAN, the most relevant work is SAIDuCANT by Olufowobi et al. They built a specification based IDS for the automotive CAN bus whose model uses the real time timing of CAN messages, and the system detected data injection attacks with few false positives compared to other timing based methods (Olufowobi et al., 2020). This shows that the specification approach fits CAN well, because CAN traffic is periodic and predictable. The same property holds for CANopen, where the allowed states, services, and timing are defined in CiA 301. A detector can encode these rules and check every frame against them. The earliest application of the idea to CANopen is again the work of Larson and co-authors, who derived their specifications directly from the CANopen profile (Larson, Nilsson, and Jonsson, 2008).

3.4. Intrusion Detection on Embedded Devices

A detection system for CANopen must run close to the bus, because in many real systems there is no powerful gateway or cloud link. The detection must happen on a small microcontroller with little memory, a slow processor, and strict real time deadlines (Reeves et al., 2011). Reeves and colleagues argued early that normal host based detection is too heavy for embedded control systems with strict timing (Reeves et al., 2011). More recent work shows that detection on a microcontroller is feasible when the model is small. Javed and colleagues embedded a tree based IDS in a smart thermostat that ran on the device itself, without a gateway, and detected attacks in a few hundred microseconds (Javed et al., 2024).

The most directly related work is by Subaşı and team. They built a real time, reconfigurable, specification driven IDS on a custom test bench that emulates an electric vehicle powertrain. Their ruleset checks timing bounds, jitter, identifier and bus load. The system met real time deadlines and a low false positive rate while staying within the limits by using automotive grade microcontrollers (Subaşı and Mercimek, 2026). This confirms that a specification based IDS can run within strict real time and resource limits on small hardware. It is close to the goal of this thesis but targets raw CAN on a vehicle powertrain, not the CANopen service layer.

3.5. Research Gap

The review leads to a gap. CAN security is a mature field with many attacks and many detection systems (Aliwa et al., 2021; Rajapaksha et al., 2023). CANopen is widely used in industry and shares all of the weaknesses of CAN plus its own at the service layer, yet the published work on practical CANopen attacks and defenses is still thin, the foundational specification based study predates modern embedded hardware (Larson, Nilsson, and Jonsson, 2008), and recent surveys cover CANopen only as one item in a larger list (Popic et al., 2025). Specification based detection is a strong match for CANopen, the approach already works for CAN timing (Olufowobi et al., 2020) and other control protocols (Lin et al., 2013; Hotellier et al., 2024). It can run on small microcontrollers within real time limits (Subaşı and Mercimek, 2026). But to the best of the knowledge gathered here, no published work builds a specification based IDS for the CANopen service layer on microcontroller class hardware and tests it against a systematic attack tool that covers the major CANopen services.

This thesis fills that gap. It builds a systematic attack tool and a specification based IDS on ESP32 hardware, so that it can test, in a controlled way, whether attacks across the major CANopen services produce distinguishable signatures and whether a specification based IDS on an ESP32 can detect them from a passive, listen-only position. The growing regulatory pressure on industrial security, through the NIS2 directive and the IEC 62443 standard, makes this work timely (Makrakis et al., 2021; Boeding, Hempel, and Sharif, 2025; International Electrotechnical Commission, 2018).

4. Methodology

This chapter describes the process used to answer the two research questions. It defines the phases of the work, states how each phase builds on the result of the one before it, and fixes the criteria used to evaluate the results against each research question. The infrastructure and the software are only named here at the level needed to follow the process. Their detailed design is given in Chapter 5, and the measured results are given in Chapter 6.

4.1. Overall Approach

The work follows a red team and blue team structure on an embedded CANopen testbed. One side, the red team, attacks the bus. The other side, the blue team, is a specification based intrusion detection system that monitors the bus and raises an alert when a frame breaks a rule of the protocol. A third node is the victim, a standard CANopen slave that is the target of the attacks.

The red team and blue team structure is an established way to evaluate a security mechanism under realistic adversarial conditions (Mirkovic, Reiher, et al., 2008; Rajendran, Jyothi, and Karri, 2011). Its value here is threefold. First, it separates the role that creates the attacks from the role that defends against them. The detection rules are written from the CANopen specification, not copied from the captured attack traffic. This keeps the evaluation honest, because the detector is tested against attacks it was not built to recognize, rather than attacks it was shaped around. The evaluation does not become circular. Second, a single set of runs produces the data for both research questions at once. When the red team executes an attack, that one event is captured twice over. The traffic it puts on the bus is the signature data that RQ1 examines, and the same traffic, fed to the detector, is what shows whether a rule fires for RQ2. Because both answers are drawn from the same events, the detection result in RQ2 is measured against exactly the attacks whose signatures were established in RQ1, with no mismatch between the two. Third, it has precedent in two different security fields, in collaborative network defense exercises (Mirkovic, Reiher, et al., 2008) and in hardware trust assessment (Rajendran, Jyothi, and Karri, 2011), which shows that the structure transfers across domains and is not specific to one kind of system. The same separation of attacker and defender is what underlies the use of dedicated testbeds in the industrial control security literature (Conti, Donadel, and Turrin, 2021).

The two research questions are answered in sequence, because RQ2 depends on the result of RQ1. RQ1 asks whether each attack leaves a distinguishable signature in the traffic. Only an attack that leaves a signature can be detected, so the attack signatures established under RQ1 are the input to the rule derivation and detection evaluation under RQ2. The work is therefore organized into four phases that run in order, and each phase consumes the output of the previous one.

4.2. Observing the Bus

Every phase of the work depends on reading the traffic on the bus, so the method of observation is fixed first. Two independent observers are used, and they serve different roles.

The blue node is the detector under test. It is an ESP32 running its TWAI controller in listen only mode, so it receives every frame on the bus but never acknowledges and never transmits. This matters for two reasons. It guarantees that the detector cannot itself alter the traffic it is measuring, and it means the detection result reflects only what a passive monitor can observe, which is the realistic deployment for a defense that must not interfere with a running machine.

The SYSWORXX USB-CAN module is an independent recorder on the same bus. It only listens and writes every frame to a file on a host computer. It plays no part in detection. The recording is the trusted record of what was actually sent on the bus during each run.

The recorder works at the raw CAN level. A CANopen message is just a CAN frame, so the SYSWORXX module does not need to know whether a frame is a heartbeat, an NMT command, or anything else. It simply captures every frame on the bus as it appears. Deciding what each frame means in CANopen terms is done later, when the recording is analyzed. This is why a generic CAN recorder is enough to serve as the ground truth.

Having this separate recorder matters. The blue node tells what the detector caught. The SYSWORXX recording tells what was actually sent. Without the second record one could not tell a real miss from a frame that never reached the bus. Because the only evidence would come from the detector itself. This is what lets the detector be scored in Section 4.5 against a count of sent frames that does not come from the detector.

4.3. Research Phases

Phase 1: Baseline characterisation

The first phase records the normal behavior of the testbed before any attack. The goal is a clean reference against which all later traffic is compared. This phase defines the four stable features of normal operation, which are the rate of each message, the length of each frame, the data pattern each frame carries, and the order of the identifiers on the bus. Without this reference there is no way to say what counts as a deviation.

Phase 2: Attack execution and signature extraction

The second phase runs the attack tool against the victim across the seven service categories and captures the resulting traffic. For each attack the captured traffic is compared against the baseline from Phase 1, and the deviation in the four baseline features is recorded as the signature of the attack. This phase produces the evidence for RQ1. Its output, the set of attack signatures, is the input to Phase 4.

Phase 3: Detection rule derivation

The third phase derives the detection rules from the CiA 301 specification and from the CiA 305 specification for the layer setting service. Each rule encodes one expected property of correct CANopen behavior, so that a frame which breaks the property is flagged. Most rules map to a specific clause of the specification, so that the detector is traceable to the standard rather than to ad hoc heuristics. Some attacks use frames that are conformant at the transport layer, so a clause alone cannot flag them. A rule for such an attack adds an observed runtime property, such as the timing or the rate of the frames on the bus. The rules are derived from the specification and from expected runtime behavior, not learned from the attack traffic. This makes the approach specification based and able to flag unseen variants. This phase produces the rule set, together with the source of each rule, which answers RQ2.

Phase 4: Detection evaluation

The fourth phase runs every attack from Phase 2 against the detection rules from Phase 3 and records, for each attack, which rule fired and which specification clause it was grounded in. Each attack is run twice, under two settings of the same detector that differ only in which rules are active. In Mode A only the transport layer rules are enabled. These rules judge a frame by how it is sent, using its timing, and the bus load. This is the kind of information a CAN IDS works with, so Mode A serves as an approximation of a transport layer detector, without claiming to reproduce any specific product. In Mode B the full CANopen aware rule set is enabled, adding the rules that judge a frame by what it means in CANopen terms. Comparing the two modes shows what the CANopen aware rules add over the transport layer rules alone. An attack that is caught in Mode B but not in Mode A is one that the transport layer rules cannot flag, and detecting it is the reason a CANopen aware detector is needed.

4.4. Success Criteria

The criteria for each research question are fixed before the runs, so that the evaluation in Chapter 6 is a check against a defined target and not a post hoc judgement.

Attack success (input to RQ1)

An attack is not counted as successful merely because frames were sent. Success is defined at two levels. The first level is effect on the victim, which means the victim does something it should not do or stops doing something it should do. An NMT attack succeeds if the victim leaves the operational state. A PDO injection succeeds if a spoofed frame with false process data reaches the bus on a genuine PDO identifier. A denial of service succeeds if normal frames can no longer meet their timing. The second level is signature, which means the attack leaves a clear and repeatable change in at least one of the four baseline features. For RQ1 the second level is the decisive one, because an attack that has an effect but leaves no measurable signature could not be detected.

RQ1 criterion

RQ1 is supported if every one of the seven attack categories produces a signature that is distinguishable from the baseline and stable across repeated runs. A signature that appears in one run but not in the next would not support the claim.

RQ2 criterion

RQ2 is answered if a detection rule can be derived for each studied attack category, and if the source of each rule is stated. A rule is grounded either in a specific clause of CiA 301 / CiA 305 or in an observed runtime property such as message timing where a clause alone is not enough. The detection evaluation of Phase 4 confirms the derivation by showing that each rule fires on the attack it targets. An attack category for which no rule can be derived at all marks the boundary of the approach rather than a failure of it.

4.5. Repeatability and Evaluation Design

The runs are repeated to confirm that each signature and its detection are stable rather than incidental. The evaluation is qualitative, so the number of repetitions is chosen to establish this

stability and not to estimate a detection rate. It does not compute inferential statistics. For each attack category it reports whether the attack produced a signature that is distinguishable from the baseline and whether the correct detection rule fired. The comparison between Mode A and Mode B in Chapter 6 rests on the same categories.

The evaluation is organized so that each attack category is run in each detector mode, and every such combination is exercised by at least five repeated runs. The scenario label of every run is written into the log files, so that each run can be traced back to its attack category and detector mode. Chapter 6 reports one representative capture per combination. Across the repeated runs the reported signature and alert were reproduced without exception, which is the basis for the repeatability claim.

5. Implementation

This chapter describes how the testbed was built. It is organized in three parts. The first part, the infrastructure, covers the hardware and the network topology. The second part, the software, covers the firmware, the CANopen stacks, the attack tool, and the detection engine. The third part, the test cases, defines the seven attacks and the detection rules that are exercised in the evaluation. The process that uses this testbed is defined in Chapter 4, and the measured outcomes are reported in Chapter 6.

5.1. Infrastructure

The infrastructure covers the testbed hardware and how the nodes are connected.

5.1.1. Overview

The testbed uses a three node topology on embedded hardware. ESP32 #1 is the *red node*, the attacker that implements the CANopen service categories. ESP32 #2 is the *blue node*, which runs the intrusion detection system and monitors the bus in listen-only mode. ESP32 #3 is the *victim node*, a standard CANopen slave that is the primary target. All three nodes share a single CAN bus. A Systec SYSWORXX USB-CAN module is attached to the same bus as a fourth, passive participant. It does not take part in the attack or the detection. It records every frame on the bus to a host computer and serves as the independent ground-truth capture defined in Section 4.2. An out-of-band Wi-Fi and MQTT channel connects the host PC to the red and blue nodes and keeps attack commands off the monitored bus.

Figure 5.1 shows the physical layout of the testbed. The red, blue, and victim nodes drive or observe a single terminated CAN bus, while the SYSWORXX module records every frame as the independent ground truth defined in Section 4.2. The blue node and the recorder connect to the bus as listen-only taps and never transmit. The host reaches the red and blue nodes only over the separate Wi-Fi and MQTT channel, so no attack command is ever visible to the detector on the bus it monitors.

Three separate ESP32 microcontrollers were chosen for the active nodes instead of a larger single board computer, a simulation, or a single ESP32 running several roles. Three reasons drove this decision. First, separate nodes constrain the detector to the same resource envelope as a real deployed node, which keeps the feasibility of the approach grounded in realistic hardware. Second, real silicon gives accurate timing measurements that reflect actual interrupt and bus arbitration behavior rather than simulation approximations. Third, placing the detector on its own microcontroller avoids any chance that the attack workload, the victim workload, and the detection workload interfere with each other inside one device.

5.1.2. ESP32 Microcontroller

All three nodes are based on the Espressif ESP32-WROOM-32 module. The ESP32 has a dual-core Xtensa LX6 processor running at up to 240 MHz, 520 KB of on-chip SRAM, and 4 MB of flash, and it includes Wi-Fi and Bluetooth. Most importantly for this thesis, it has a built-in Two-Wire Automotive Interface (TWAI) controller, which is compatible with ISO 11898-1

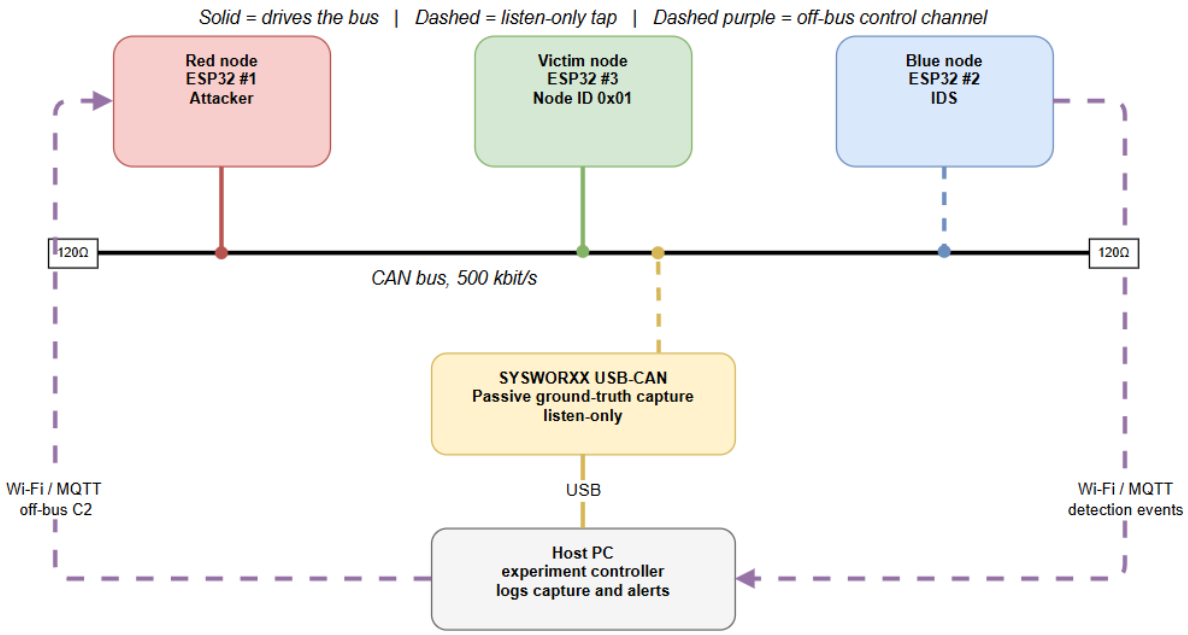

Figure 5.1.: Testbed topology.

CAN 2.0B and supports bit rates up to 1 Mbit/s. The TWAI controller can operate in normal, self-test, or listen-only mode. The blue node uses listen-only mode so that it can monitor the bus without sending acknowledgement bits and without affecting bus timing. The listen-only initialisation is shown in Listing A.1 in Appendix A.

The 520 KB of SRAM and 4 MB of flash leave the detection logic constrained to the resource budget of a small industrial module, so the device is a conservative choice for studying the approach on microcontroller.

5.1.3. CAN Transceivers

The TWAI controller is a digital peripheral and needs an external transceiver to drive and receive the differential bus signals. All three nodes use the Texas Instruments SN65HVD230, which implements the ISO 11898-2 CAN physical layer, operates at 3.3 V to match the ESP32 I/O voltage, and supports bit rates up to 1 Mbit/s. It converts the single-ended TX and RX signals of the ESP32 into the CAN_H and CAN_L differential pair.

The two wires of the pair carry the same logical signal as a voltage difference rather than against ground. A recessive bit holds both wires near a common level, and a dominant bit drives CAN_H high and CAN_L low, so the bit value is read from the difference between the two and not from either wire alone. This differential scheme is what gives CAN its noise immunity, because interference couples into both wires almost equally and cancels in the difference, which matters in the electrically noisy industrial settings where CANopen runs.

The three transceivers are connected by a short twisted-pair cable of approximately 20 cm. The bus is terminated at the SYSWORXX end with a 120 Ω resistor housed in the DE-9 connector, following the 120 Ω termination specified in ISO 11898-2 (International Organization for Standardization, 2016). At this length and at the 500 kbit/s rate used in all experiments, cable propagation delay is negligible, so the resulting reflections settle well within the bit period and bus timing is determined by the TWAI peripheral and the transceiver electronics.

Figure 5.2 shows the full wiring of the testbed. The red, victim, and blue nodes share one

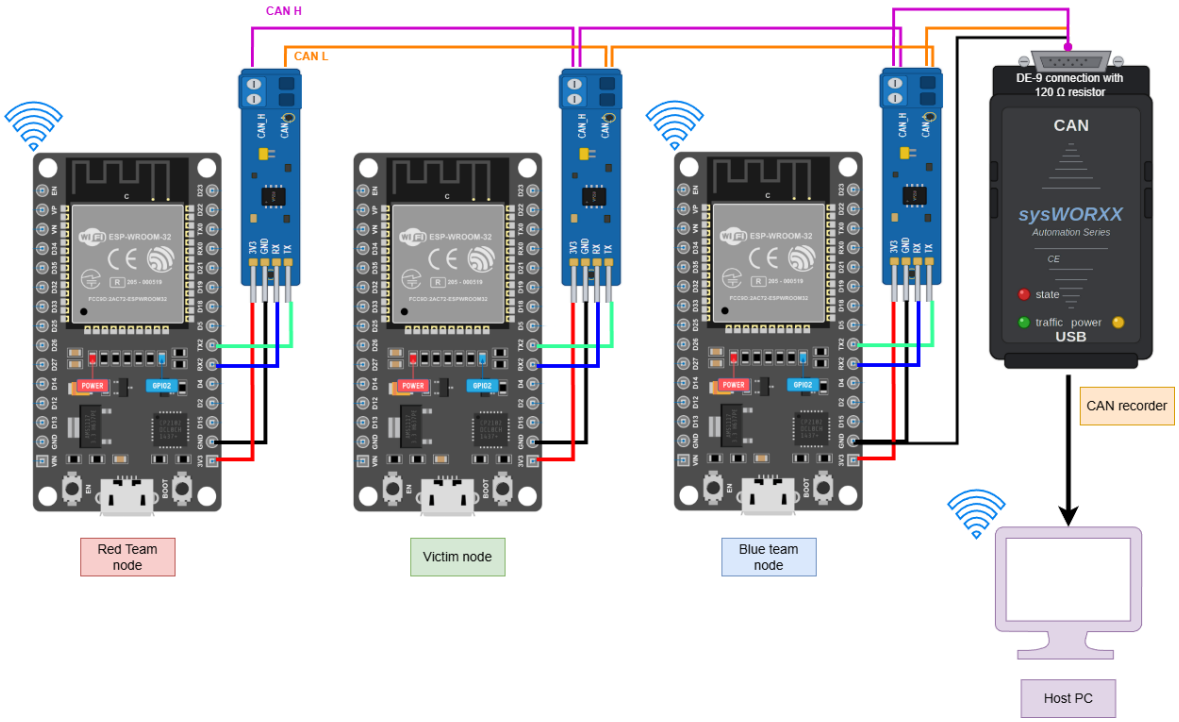

Figure 5.2.: Testbed cabling.

bus, and the SYSWORXX module listens passively and writes every frame to the host PC as the ground-truth record. The Wi-Fi links to the host are drawn apart from the bus, showing that the out-of-band control and logging path places no traffic on the CAN bus itself.

A bit rate of 500 kbit/s was used in all experiments. It is one of the standard bit rates defined for CANopen and is widely used in industrial networks, which makes the testbed representative of realistic deployments (CAN in Automation (CiA), 2011).

5.1.4. Host PC and Out-of-Band Channel

A host PC serves as the experiment controller. The SYSWORXX CAN module connects to this host and records the complete CAN traffic on the bus. Because a CANopen message is just a raw CAN frame, the module captures every frame without needing to know which CANopen service it belongs to. The interpretation of each frame is done later when the recording is analyzed. The host also connects to the red and blue nodes over Wi-Fi using MQTT, with each node on its own topic. It acts as a command and control unit for the red node, which receives JSON attack commands and translates them into CAN frame sequences. The blue node reports detection events back to the host, which writes them to log files. Attack sequences are scripted in Python and run without manual interaction.

The out of band MQTT channel is an intentional design choice. Attack commands never appear on the CAN bus, so the blue node cannot detect an attack by watching control traffic. It can only detect the malicious CAN frames that the attack produces. This mirrors a real scenario in which an attacker uses a separate channel to deliver payloads without exposing the command infrastructure to the monitored medium.

Wi-Fi was chosen as the medium for this channel because it is separate from the CAN bus and is already built into the ESP32, so no extra hardware is needed to keep the control path

off the monitored medium. Each node runs in station mode and connects to a single access point, which places the host and the nodes on one wireless network without adding routing or gateway components to the testbed.

MQTT was chosen as the protocol on top of Wi-Fi for three reasons. First, it uses a publish and subscribe model with named topics. Each node has its own topic and the host addresses the red node and the blue node independently without point to point connections. The host publishes a JSON attack command to the topic `canopen/red/cmd`, and the blue node publishes its detection events on its own topic, so command delivery and event reporting are fully decoupled. Second, this topic structure makes the runs easy to script in Python and to repeat without manual interaction, which is what produces the reproducible evaluation runs. Third, MQTT is a lightweight protocol designed for small networked devices, so it fits the ESP32 and adds little overhead to the node that is also running the attack or the detection logic.

5.2. Software

This section describes the software running on the testbed. It covers the firmware framework shared by all nodes, the CANopen stack that gives the victim its protocol behavior, the attack tool on the red node, and the detection engine on the blue node. It closes with the threat model that defines what the attacker is assumed to be able to do.

5.2.1. Firmware Framework

All nodes run C firmware built with ESP-IDF v5.5.1. ESP-IDF provides FreeRTOS as the real-time kernel, a TWAI driver with interrupt-driven frame reception, a Wi-Fi stack with an MQTT client, and hardware timers with microsecond resolution. ESP-IDF was chosen because it exposes the TWAI listen-only mode and the hardware acceptance filter registers directly (Espressif Systems, 2025b), and because it gives deterministic task scheduling through FreeRTOS priorities (Espressif Systems, 2025a), which keeps the detection task responsive to incoming frames.

5.2.2. CANopen Stack

The CANopen slave stack on the victim node is a lightweight custom library rather than a commercial stack. A minimal custom implementation gives precise control over which CiA 301 behaviors are active, which makes it possible to confirm that a rule trigger corresponds to a genuine protocol violation rather than a quirk of a third-party library. A custom library also avoids licensing constraints and keeps the source code available for inspection and reproducibility. The blue node carries only the passive CANopen knowledge it needs to interpret frames, such as the expected heartbeat period of the victim, and it never acts as an active master on the bus.

In Table 5.1 the objects are listed that are mandatory for a conformant CANopen device in the role of the victim node. The upper part lists the communication objects required by CiA 301, together with the single application data object used as the PDO payload during testing. The lower part lists the two default transmit process data objects for node-ID `0x01`. TPDO1 transmits at COB-ID `0x181`, which is the default identifier `0x180` plus the node-ID. TPDO2 transmits at COB-ID `0x281`, the default identifier `0x280` plus the node-ID. Both carry the application data object as their payload. This minimal set is sufficient to exercise the seven attack categories. Each attack targets one of these objects.

TPDO1 is set to 100 ms and TPDO2 to 500 ms so the bus carries one fast and one slow cyclic PDO. This gives the timing rules two distinct periods to check and makes the order of

identifiers a usable feature. The heartbeat runs at 1000 ms, which is slower than both PDOs, so the three cyclic message types sit at three clearly separated rates.

Index / COB-ID	Name	Role
`0x1000`	Device Type	Device profile identification
`0x1001`	Error Register	Error status flags
`0x1014`	EMCY COB-ID	Emergency message identifier
`0x1017`	Heartbeat Producer Time	Heartbeat production interval
`0x1018`	Identity Object	Vendor and device identity
`0x181`	TPDO1	Transmit (victim to bus)
`0x281`	TPDO2	Transmit (victim to bus)

Table 5.1.: Object dictionary entries and PDOs implemented on the victim node.

5.2.3. Red Node Attack Tool

The red node runs the attack tool. Its attack logic covers the seven service categories: NMT state manipulation, SDO abuse, PDO injection, heartbeat manipulation, EMCY injection, LSS abuse, and bus-level denial of service. The red node subscribes to the MQTT topic `canopen/red/cmd`, parses each JSON payload into a CAN frame sequence built according to the CANopen encoding rules, and transmits it through the TWAI peripheral. Each command specifies the attack category, the variant ID, the target node ID, and optional parameters such as injection rate or SDO index.

5.2.4. Blue Node Detection Engine

The blue node runs the detection engine with the TWAI peripheral in listen-only mode. A background task reads each frame from a queue and dispatches it by COB-ID to the matching service check. The bus load check, runs for every frame regardless of identifier, because a denial of service attack does not respect the service map. When a check decides that a frame breaks the specification, it fills an alert structure with a rule name and the suspect COB-ID. The alert is stored in an SRAM ring buffer and published over MQTT to the host, where it is grouped by attack category and counted. The blue node never transmits on the bus. It only observes and reports, which keeps the approach a pure detection system and guarantees that the monitor cannot alter the traffic it measures.

Every frame is timestamped on the ESP32 clock when it is received, and the detection task runs on a dedicated core so that frame reception and rule evaluation do not compete with other work on the device. This timestamp is the basis for all timing checks.

The detector keeps a whitelist of the node IDs commissioned on the bus. In this testbed the whitelist holds only the victim node `0x01`. The whitelist is the anchor for many rules, because a frame that claims a node ID outside it is by itself a strong signal of an intruder.

The blue node carries a two-mode configuration flag, which is the structural mechanism for measuring what the CANopen aware rules add. The two modes are two settings of the same detector that differ only in which rules are active. Mode A enables only the transport layer rules, which judge a frame by its identifier and its rate and which represent the kind of information a CAN IDS works with, without claiming to reproduce any specific product. Mode B enables the full CANopen aware rule set, which adds the rules that judge a frame by what it means in CANopen terms. Each attack is run against both modes, and an attack caught in Mode B but not in Mode A is one that the transport layer rules cannot flag.

Table 5.2 lists the rules of the detection engine, grouped by CANopen service. The NMT rules flag state commands that no legitimate master should send and command specifiers that are not valid. The SDO rules treat any access during idle operation as a signal and turn abort responses and request floods into evidence. The PDO rules flag process data from an unknown node ID and frames that arrive earlier than their expected period. The heartbeat rules detect a heartbeat from an unknown node, a duplicate heartbeat for an existing node, and an invalid state value. The EMCY rules cover emergency frames and known error codes. The LSS rule treats any LSS master frame during operation as abnormal, since commissioning is not expected at runtime. The bus load rules count frames per COB-ID and across the whole bus, and fire when either count passes its threshold. The bus load rules run for every frame and stay active in both modes, because a denial of service attack does not respect the service map. The full detection engine is available in the repository, see Appendix A. Listing A.2 and Listing A.3 there show a specification-derived and a runtime-derived rule.

Rule	Service	Condition that fires the alert
R_NMT_CONTROL	NMT	Unauthorised NMT state command
R_NMT_BAD_CS	NMT	Invalid NMT command specifier
R7a, R7b, R7c	SDO	SDO access or abort during idle operation
R8	SDO	SDO request flood above threshold
R_PDO_UNKNOWN_NODE	PDO	PDO from a node ID outside whitelist
R_PDO_INJECT	PDO	PDO arriving earlier than expected
R6a_GHOST_NODE	Heartbeat	Heartbeat from a node ID outside whitelist
R6b_HEARTBEAT_SPOOF	Heartbeat	Duplicate heartbeat for an existing node
R6c_HEARTBEAT_STATE	Heartbeat	Heartbeat reporting an invalid state value
R1–R5	EMCY	EMCY frames and known error codes
R_LSS_ABUSE	LSS	Any LSS master frame during operation
R_BUS_FLOOD_ID	Bus load	Frames per single COB-ID above threshold
R_BUS_FLOOD_TOTAL	Bus load	Total frames per second above threshold

Table 5.2.: Detection rules of the blue node engine.

5.2.5. Threat Model

The threat model follows the CANopen attack surface analysis of Larson, Nilsson, and Jonsson, the foundational reference for the security properties of this protocol (Larson, Nilsson, and Jonsson, 2008). The model assumes that the adversary already has bus-level access, meaning the ability to inject arbitrary CAN frames. The attacker may aim to disrupt network management through NMT attacks, to read or corrupt process data through SDO and PDO attacks. To suppress status monitoring by manipulating heartbeats, to inject false fault reports through EMCY, or to saturate the bus through denial of service. The victim is the primary target of these goals.

This access assumption is realistic, because industrial CANopen networks are often physically accessible in factories, maintenance bays, and building automation, and any node that reaches the bus can impersonate any other node. Pfeiffer made this point specifically for CANopen. An intruder with bus access can flood high priority COB-IDs and make the network unusable, and the only effective countermeasure is detection before the bus is saturated (Pfeiffer, 2017).

The model excludes several attack types. It excludes physical layer attacks, such as cutting bus wires or manipulating voltage levels. It excludes attacks that require prior firmware compromise of a node. It also excludes remote attacks that would need a gateway between

the CAN bus and an external IP network, since no such gateway exists in the testbed. These exclusions match the research scope. The scope is application layer CANopen attacks and the detectability of their CAN-bus signatures.

5.3. Test Cases

This part defines the seven test cases. Each test case is one attack category. For each category it states the goal of the attack, the frames the red node sends, the detection rules the blue node applies, and the logic by which those rules decide. The captured traffic and the alerts that these test cases produce are not shown here. They are reported in Chapter 6.

A note applies to the service based attacks. Some CANopen services are not cyclic and appear on the bus only when a transfer or a command happens. The SDO and LSS services are of this kind, so their identifiers never appear in the idle baseline and appear only when a client or master starts an exchange. For these services the first sign of an attack is the simple appearance of a frame that the baseline never shows.

5.3.1. NMT State Manipulation

Goal. The goal is to change the operational state of the victim against the intent of the network. The NMT service controls the state of each node, and a valid NMT command can move a node to stopped or to pre-operational. In stopped state the node sends almost no traffic, and in pre-operational state it stops its PDOs but still answers SDOs and sends heartbeats. An attacker that controls the state can stop a working node from doing its job.

Frames sent. The attack sends an NMT command frame on COB-ID `0x000` with two data bytes. The first byte is the command specifier and the second is the target node ID, here `0x01`. The specifier `0x80` requests pre-operational, `0x02` requests stopped, and `0x81` requests a reset of the node.

Rules applied. Two rules apply. Rule `R_NMT_CONTROL` fires on any valid NMT command and names the command from its specifier. Rule `R_NMT_BAD_CS` fires when the specifier is not a defined NMT command, which marks a malformed or crafted frame.

Detection logic. The detector decodes the specifier against the CiA 301 set: start (`0x01`), stop (`0x02`), enter pre-operational (`0x80`), reset node (`0x81`), and reset communication (`0x82`). A stop, a reset node, or a reset communication is raised as critical, because these take the victim out of operation. A start or an enter pre-operational is raised as a warning. A specifier outside the defined set is raised as a malformed frame. The detection does not depend on a victim reaction, because the identifier `0x000` carries no normal traffic during operation, so the presence of the control frame is itself the signal.

5.3.2. SDO Abuse

Goal. The goal is to misuse the SDO protocol to read or write the object dictionary of the victim without authority. An attacker can try to read values it should not see, write values it should not change, or flood the victim with requests to stress it and the bus.

Frames sent. The attack sends SDO requests on the client identifier `0x601`. A request carries a command byte, an index, a sub-index, and data. A read uses an upload request, a write uses a download request, and the flood sends many requests in a short time. The four variants exercised are an unauthorized read of object `0x1017`, an unauthorized write of object `0x6001`, a refused write to the read-only object `0x1017` and a high-rate flood of reads on object `0x6000`.

Rules applied. Four rules apply. Rule `R7a_SDO_UNKNOWN_NODE` fires when an SDO targets a node ID outside the whitelist. Rule `R7c_SDO_ACCESS` fires on any unsolicited read or write during operation. Rule `R7b_SDO_ABORT` fires when the server returns an abort, which proves a refused access attempt. Rule `R8_SDO_FLOOD` fires when the rate of SDO requests for a node passes the threshold.

Detection logic. The SDO request uses COB-ID `0x600` plus the node ID. The response uses `0x580` plus the node ID. The first data byte is the command specifier. The next two bytes hold the object index in little endian order. A specifier in the `0x2x` range is a write. A specifier of `0x40` is a read. A response specifier of `0x80` is an abort.

Three rules act on these frames. The access rule treats a single SDO transfer during idle operation as a signal, because the baseline shows no SDO traffic at all. The abort rule treats a refusal as evidence of an attempt. The flood rule counts requests in a one second window and fires when the count passes ten.

These rules also catch the stealth case. Here a node manipulates the object dictionary without announcing itself through a heartbeat. The rules act on the SDO frames directly. They do not depend on a heartbeat from the source.

5.3.3. PDO Injection

Goal. The goal is to inject false process data onto the bus. The PDO service carries the real-time data of the network and has no check on the sender, so an attacker can send frames with the COB-ID of a real PDO and any consuming node may accept the false data. This is a spoofing attack at the application layer (Zhang, Meng, Yan, et al., 2020; Oladimeji et al., 2023).

Frames sent. The attack sends frames on `0x181` and `0x281`, the transmit PDOs of the victim. The payload is eight bytes of chosen data, and the attacker can copy the structure of the real PDO while setting false values. Two variants are exercised, a single spoofed frame and a continuous flood.

Rules applied. Two rules apply. Rule `R_PDO_UNKNOWN_NODE` fires when a PDO carries a node ID outside the whitelist, which marks a spoofed source. Rule `R_PDO_INJECT` fires when a PDO arrives too soon after the previous frame on the same identifier, which marks an injected frame.

Detection logic. The transmit PDOs run at fixed periods, TPDO1 on `0x181` at 100 ms and TPDO2 on `0x281` at 500 ms. A single genuine producer cannot send two frames on one identifier closer together than its period. The detector records the arrival time of each PDO and measures the gap to the previous frame on the same identifier. When the gap falls below half of the nominal period, the frame cannot come from the genuine producer alone and is flagged as injected. This is the timing signal that the grouped capture cannot show, because the grouped view folds the genuine and injected frames into one row, whereas the detector reads the bus as a linear stream and sees the two frames as separate arrivals.

5.3.4. Heartbeat Manipulation

Goal. The goal is to attack the health signal of the network. The heartbeat tells other nodes that a node is alive and in which state. Two abuses are studied. Suppression hides or removes a real heartbeat. Spoofing sends a heartbeat that claims a false state or a node that does not exist. A spoofed heartbeat for a missing node is a ghost node, and a spoofed heartbeat that claims a healthy state for an unhealthy node is a false node-alive.

Frames sent. The attack sends frames on a heartbeat identifier, which is `0x700` plus a node ID. For the victim this is `0x701`, and for a ghost node it is the heartbeat identifier of a node ID with no real node, here `0x77E` for node `0x7E`. The single data byte sets the claimed state, with `0x05` for operational and `0x7F` for pre-operational.

Rules applied. Four rules apply. Rule `R6a_GHOST_NODE` fires on a heartbeat from a node ID outside the whitelist. Rule `R6b_HEARTBEAT_SPOOF` fires when two heartbeats for one node arrive closer than a single producer could send them. Rule `R6c_HEARTBEAT_STATE` fires when the heartbeat carries an invalid state byte. Rule `R6_HEARTBEAT_LOST` fires when an expected heartbeat does not arrive within the timeout.

Detection logic. The ghost node rule is the direct case, because a heartbeat on an identifier that maps to a node ID not in the whitelist is an announced node that does not exist.

The spoof rule handles the harder case where the attacker reuses the identifier of a real node. A single producer holds one fixed period, so two heartbeats for the same node arriving closer than the minimum interval must come from two sources, and the second is the spoof. This rule works even when the spoofed state copies the true state, because it acts on timing and not on content. The false state rule checks the state byte against the valid CiA 301 set of bootup, stopped, operational, and pre-operational.

The loss rule runs on a periodic timer and fires when the time since the last heartbeat of a whitelisted node passes the timeout, which catches suppression, where no frame exists for a content rule to inspect.

5.3.5. EMCY Injection

Goal. The goal is to inject false emergency messages. The EMCY service reports faults, and other nodes or operators may react to them. A false emergency can trigger a wrong reaction, and a flood of emergencies can bury a real fault in noise.

Frames sent. The attack sends EMCY frames on `0x080` plus a node ID, here `0x081` for the victim. Bytes zero and one hold the emergency error code in little endian order, and byte two holds the error register. The attacker sets these to claim a fault that did not occur or a severity that does not match the node state. Three variants are exercised, a single generic fault, a single overvoltage fault, and a flood.

Rules applied. Five rules apply. Rule `R1_UNKNOWN_NODE_EMCY` fires on an emergency from a node ID outside the whitelist. Rule `R2_EMCY_FLOOD` fires when the emergency rate for a node passes the threshold. Rule `R3_EMCY_TIMING` fires when two emergencies from one node arrive closer than the minimum interval. Rule `R4_BAD_ERROR_CODE` fires on an implausible error code. Rule `R5_SIGNATURE_MATCH` fires when the manufacturer bytes match the red node signature.

Detection logic. The unknown node rule is the strongest, because the EMCY identifier carries no traffic in the idle baseline, so an emergency from a node ID the network does not contain is a clear intruder.

The flood rule counts emergencies in a one second window with a threshold of three, and the timing rule measures the gap between consecutive emergencies. Together they catch a stream of false faults.

The error code rule flags error codes that a normal device does not send. Each EMCY frame carries an error code, and some code ranges are never used in normal operation. A frame that uses one of these ranges is therefore suspect. The signature rule is the most exact. The red node writes a fixed marker into the manufacturer bytes of every EMCY frame it sends. When the detector sees this marker, it knows the frame came from the attacker.

The rate and timing rules catch a different sign. A real fault is rare and happens on its own. So if a node sends many EMCY frames while its heartbeat still reports operational and its PDOs keep running, something is wrong. The node claims a fault but behaves as if healthy. This contradiction is what the rate and timing rules turn into an alert.

5.3.6. LSS Abuse

Goal. The goal is to abuse the layer setting services, which let a master change low-level settings such as the node ID and the bitrate. A wrong bitrate can cut a node off the bus, and a changed node ID can break the addressing of the whole network.

Frames sent. The attack sends LSS frames on the LSS master identifier `0x7E5`. The LSS protocol needs two steps. The attacker first switches the slave into configuration mode and then sends the configuration command. The first byte of each frame is the command specifier, with switch mode global (`0x04`), configure node ID (`0x11`), and configure bitrate (`0x13`). Two variants are exercised, one that sets a new node ID and one that sets a new bitrate.

Rules applied. One rule applies. Rule `R_LSS_ABUSE` fires on any LSS master frame during operation and names the service from its command specifier.

Detection logic. The detector names the common services, which are configure node ID (`0x11`), configure bitrate (`0x13`), switch mode global (`0x04`), store configuration (`0x15`), and activate bitrate (`0x17`). The detection does not depend on a victim reaction, because the identifier `0x7E5` carries no traffic in a running network, so any LSS frame during operation is the signal. This is the cleanest test case, because LSS runs only at commissioning and never during normal traffic. It also flags a bitrate change before it can take effect and cut the victim off the bus.

5.3.7. Bus-Level Denial of Service

Goal. The goal is to deny service by flooding the bus. CAN uses priority based arbitration, so a frame with a lower identifier wins the bus, and an attacker that sends many high priority frames can take the bus from all other nodes. This attack does not target one service. It targets the shared medium (Murvay and Groza, 2017; Popic et al., 2025).

Frames sent. The attack sends frames at a very high rate with the lowest identifier `0x000`, which wins arbitration against the victim PDOs and heartbeat. The key parameter is the inter frame gap, where a smaller gap means a higher load.

Rules applied. Two rules apply. Rule `R_BUS_FLOOD_ID` fires when a single identifier passes the per-identifier frame threshold in a one second window. Rule `R_BUS_FLOOD_TOTAL` fires when the total frame count across all identifiers passes the total threshold in a one second window.

Detection logic. This check runs for every frame, before the service dispatch, because the flood does not respect the service map. The detector keeps a frame count per identifier and a total count across the bus, each in a one second window, and the thresholds are set well above the busiest legitimate identifier so that normal cyclic traffic does not trigger them. This check sits in the transport layer, so it runs in both Mode A and Mode B, which is fitting because denial of service is the one attack that a transport layer approach can also see. The check also marks the boundary of the specification based approach. The detector can flag the flood and the foreign identifier, but it cannot stop the arbitration damage, because that damage happens at the data link layer before any higher layer logic runs.

6. Results and Evaluation

This chapter reports the measured outcomes of the testbed. It first records the baseline of normal traffic, which is the reference for every later comparison. For each attack, it reports the signature the attack left in the traffic. It then reports the alert the detector raised in response. The attack signatures answer RQ1, and the detection outcomes feed RQ2. The chapter closes by drawing the per-category results together against the two research questions.

Each attack category was run at least five times per detector mode. The captures and alerts shown in this chapter are representative of those runs. The reported signature and alert were reproduced in every run. The evaluation reports whether each attack produced its signature and whether the correct rule fired, and does not compute a numerical detection rate.

6.1. Baseline of Normal Traffic

The baseline was captured with the victim node alone on the bus and a Systec SYSWORXX module listening passively. The victim uses node ID `0x01` and follows CiA 301, so its identifiers follow the standards listed in Section 1.2. The capture shows three cyclic message types, listed with their timing in Table 6.1.

CAN-ID	Type	Period (config)	Period (measured)
`0x181`	TPDO1	100 ms	about 100 ms
`0x281`	TPDO2	500 ms	about 500 ms
`0x701`	Heartbeat	1000 ms	about 1000 ms

Table 6.1.: Observed CANopen message types and their timing in the baseline.

The baseline establishes the four stable features used throughout the evaluation. The first is rate. Each message type arrives at its own fixed period, and the measured periods match the firmware settings, with small variations that come from arbitration and software jitter. The second is length. The heartbeat carries one byte and the two PDOs each carry eight, and the length never changes. The third is the data pattern. The heartbeat byte is `0x05`, which is the operational state, and the PDO payloads follow a regular counting pattern. The fourth is the order of identifiers, set by the periods. In one second TPDO1 sends about 10 frames, TPDO2 sends 2, and the heartbeat sends 1. The cyclic and predictable nature of this traffic is the property that makes specification based detection possible (Jiang et al., 2021; Groza and Murvay, 2019; Dönmez, 2021).

A second capture confirmed that the blue node is a true passive monitor. With the blue node attached but the victim absent, no frames appeared. This shows the detector sends no traffic of its own. With both nodes attached, the traffic matched the victim-only baseline. The monitor is therefore neutral. Any change measured in the attack runs comes from the attack and not from the detector. This is a precondition for the timing based results (Olufowobi et al., 2020).

6.2. NMT State Manipulation

The NMT case built three attacks that drive the victim into pre-operational, stopped, and reset. This section shows the trace each command left on the bus and how the detector reacted to it.

6.2.1. Attack Signature

This section reports the captured traffic before and after each NMT command. Each table lists the relevant rows from the SYSWORXX monitor. A row holds the frame count, the COB-ID, the data length, and the first three data bytes.

Baseline before any attack. Table 6.2 shows the capture before any command was sent. The victim was in the operational state. The heartbeat on `0x701` carried the state byte `0x05`. Both transmit PDOs were live. The PDO on `0x281` appeared at its normal rate. The PDO on `0x181` appeared with a high count. This matches the expected operational behavior and forms the reference for the attack runs.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
54	`0x701`	1	`0x05`	Heartbeat, operational
107	`0x281`	8	`0x64 0x00 0x69`	Transmit PDO live
530	`0x181`	8	`0x71 0x02 0x0d`	Transmit PDO live

Table 6.2.: Baseline capture before any NMT command.

After pre-operational command. The attacker sent one frame on COB-ID `0x000` with two bytes `0x80 0x01`. The first byte `0x80` is the command specifier for enter pre-operational. The second byte `0x01` is the target node ID. So the frame told node `0x01` to enter pre-operational. Table 6.3 shows the capture. A new line on `0x000` appeared with the command frame. The heartbeat state byte changed to `0x7F`, which marks the pre-operational state. The PDOs stopped producing new frames. This is the expected result. The node stops its PDOs in pre-operational but keeps sending heartbeats.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
137	`0x701`	1	`0x7f`	Heartbeat, pre-operational
223	`0x281`	8	`0x64 0x00 0xdd`	PDO stopped, stale row
1114	`0x181`	8	`0x35 0x01 0x55`	PDO stopped, stale row
1	`0x000`	2	`0x80 0x01`	Pre-operational command to node `0x01`

Table 6.3.: Capture after the pre-operational command.

After stop command. The attacker sent one frame on COB-ID `0x000` with two bytes `0x02 0x01`. Table 6.4 shows the capture. The command line on `0x000` was updated. Its byte 0 value changed from `0x80` to `0x02`. The SYSWORXX monitor shows the last frame seen on each COB-ID, so the same line now holds the stop command. The byte `0x02` is the command specifier for stop. The heartbeat byte changed to `0x04`, which marks the stopped state. The heartbeat still changed because a stopped node still sends its heartbeat by the standard. Only PDOs, SDOs, and other services are silenced in the stopped state. The PDO lines stayed

visible because the SYSWORXX monitor keeps the last seen frame and its count. These were old frames and not new traffic.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
215	0x701	1	0x04	Heartbeat, stopped
223	0x281	8	0x64 0x00 0xdd	Stale PDO row
1114	0x181	8	0x35 0x01 0x55	Stale PDO row
2	0x000	2	0x02 0x01	Stop command to node 0x01

Table 6.4.: Capture after the stop command.

After reset node command. The attacker sent one frame on COB-ID 0x000 with two bytes 0x81 0x01. Table 6.5 shows the capture. The same 0x000 line was updated again to byte 0 value 0x81. The byte 0x81 is the command specifier for reset node. After a reset the node restarts and returns to pre-operational. The heartbeat byte was 0x7F, which is pre-operational, so the heartbeat kept changing. The PDO lines again stayed unchanged.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
263	0x701	1	0x7f	Heartbeat, pre-operational
223	0x281	8	0x64 0x00 0xdd	Stale PDO row
1114	0x181	8	0x35 0x01 0x55	Stale PDO row
3	0x000	2	0x81 0x01	Reset node command to node 0x01

Table 6.5.: Capture after the reset node command.

Interpretation. Table 6.6 summarizes the heartbeat response to each command. The heartbeat byte followed every command. The attacker drove the victim through pre-operational, stopped, and reset states with single frames. This proves control over the node state and meets the success criterion. One caveat applies to the PDO rows. The SYSWORXX monitor keeps the last seen frame, so a static PDO row does not prove live traffic. For the RQ1 feature extraction the PDO stop must be confirmed from timestamps or frame counts.

Command	Specifier	State byte	Resulting state
Pre-operational	0x80	0x7F	Pre-operational
Stop	0x02	0x04	Stopped
Reset node	0x81	0x7F	Pre-operational

Table 6.6.: Heartbeat state byte after each NMT command.

The attack touches three of the four baseline features at once. It changes the identifier order, because the PDOs vanish from active transmission. It changes the rate, because the PDO rate drops to zero. It changes the data pattern, because the heartbeat state byte flips from 0x05 to 0x7F or 0x04. This makes the NMT attack the clearest case for RQ1.

6.2.2. Detection

The NMT attacks were run against the blue node while it listened on the bus, and each command produced an alert on the host SOC console. The console prints alerts as shown in Table 6.7. This console output is a proof of concept. It shows that the rule fired and what

it found, which is enough to demonstrate detection. A production system would not rely on a printed console. It would forward the alerts to a logging or SIEM system, so they can be stored and tracked over time.

Detection of the enter pre-operational command. The node read the command on `0x000` as enter pre-operational, targeting the victim. The rule `R_NMT_CONTROL` fired and raised the alert at the warning level, shown in Table 6.7. It was decided to treat this as a warning, because the command moves the node into pre-operational and does not stop or reset it. In some systems this can be part of normal operation, so a warning is a reasonable default. In others it could be unwanted and should be treated as an error. The right level depends on how the system is used, so this decision is left to the OT security team that operates the network.

Field	Value
Severity	WARNING
Category	NMT abuse
Rule	`R_NMT_CONTROL`
Suspect node	`0x01`
Suspect COB	`0x000`
Command	enter pre-op, specifier `0x80`

Table 6.7.: SOC alert for the NMT enter pre-operational command.

Detection of the stop command. The node read the command as stop, and the rule `R_NMT_CONTROL` fired again, this time at the critical level, shown in Table 6.8. The severity rose because the stop command takes the victim out of operation, which is a direct attack on the availability of the node. The same rule handles both the pre-operational and the stop command, and the severity comes from the command specifier, not from a separate rule. One rule can therefore cover the whole NMT control service while still grading the danger of each command.

Field	Value
Severity	CRITICAL
Category	NMT abuse
Rule	`R_NMT_CONTROL`
Suspect node	`0x01`
Suspect COB	`0x000`
Command	stop, specifier `0x02`

Table 6.8.: SOC alert for the NMT stop command.

Detection of the reset node command. The node read the command as reset node, and the rule fired at the critical level, shown in Table 6.9. This command is the most disruptive of the three commands, because it does not only change the state of the node, it restarts it, so any state the node held is lost. It is graded critical, the same level as the stop command, because both deny service, and a reset goes further by clearing the node state.

Summary of the NMT defense. The three captures show that the node detected every NMT control command and graded each one by its danger. Enter pre-operational was raised as a warning, while stop and reset node were both raised as critical. In all three cases the single

Field	Value
Severity	CRITICAL
Category	NMT abuse
Rule	`R_NMT_CONTROL`
Suspect node	`0x01`
Suspect COB	`0x000`
Command	reset node, specifier `0x81`

Table 6.9.: SOC alert for the NMT reset node command.

rule `R_NMT_CONTROL` fired, decoded the command, and named the target node. The second rule `R_NMT_BAD_CS` did not fire, because no malformed frame was sent in this run. The detection did not depend on any victim reaction. The presence of a frame on `0x000` was the signal, because the identifier carries no normal traffic during operation.

6.3. SDO Abuse

The SDO case built four attack variants, from a single unauthorized read to a request flood. This section shows the trace each one left on the bus and how the detector reacted to it.

6.3.1. Attack Signature

In all four SDO variants the identifiers `0x601` and `0x581` appeared on the bus, and in the baseline they never appear. This presence alone is the first and most reliable signal of SDO abuse. The heartbeat on `0x701` stayed at `0x05` throughout, so the victim never left operation during any SDO attack. Tables 6.10 to 6.13 show one capture per variant, with the heartbeat and the cyclic PDOs continuing in the background.

The read request on `0x601` in Table 6.10 is an upload request for index `0x1017`. An upload is the SDO term for a read, where the client fetches a value from an object of the server. The victim answers on `0x581` with an upload response that carries the value, and the matching index proves the answer belongs to this read. The attacker thus reads an internal configuration value without authority. The count of one on both identifiers shows the read happened exactly once while the PDOs kept running.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
1170	`0x701`	1	`0x05`	Heartbeat, operational
1	`0x601`	8	`0x40 0x17 0x10`	SDO upload request
1	`0x581`	8	`0x4b 0x17 0x10`	SDO upload response
2341	`0x281`	8	`0x64 0x00 0x83`	TPDO2, cyclic
11702	`0x181`	8	`0x40 0x02 0x90`	TPDO1, cyclic

Table 6.10.: Capture of the unauthorised SDO read on object `0x1017`.

The write request on `0x601` in Table 6.11 is an SDO download to index `0x6001`. A download is the SDO term for a write, where the client sends a value into an object of the server. The victim confirms the write on `0x581`. It thus changed its own object dictionary on the command of an unauthorized node. The process data on `0x281` then carries the marker `0xef 0xbe 0xef`, a value placed by the attack.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
1224	0x701	1	0x05	Heartbeat, operational
2	0x601	8	0x2b 0x01 0x60	SDO download request
2	0x581	8	0x60 0x01 0x60	SDO download response
2449	0x281	8	0xef 0xbe 0xef	TPDO2, cyclic
12241	0x181	8	0xd7 0x00 0xab	TPDO1, cyclic

Table 6.11.: Capture of the unauthorised SDO write to object 0x6001.

The refused write in Table 6.12 is a download to index 0x1017, but the victim answers on 0x581 with an SDO abort. The object is read-only, so the abort proves that the victim enforces the access rights of its object dictionary. The abort is a useful detection feature, because it marks an SDO transfer that was tried and denied.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
1269	0x701	1	0x05	Heartbeat, operational
3	0x601	8	0x2b 0x17 0x10	SDO download request
3	0x581	8	0x80 0x17 0x10	SDO abort response
2538	0x281	8	0xef 0xbe 0x48	TPDO2, cyclic
12690	0x181	8	0x98 0x02 0x6c	TPDO1, cyclic

Table 6.12.: Capture of the refused SDO write to read-only object 0x1017.

The flood in Table 6.13 repeats the read on index 0x6000, and the count is what separates it from the single read. The earlier read showed a count of one, while here both identifiers reach 1141, each request answered by the victim. The high and nearly equal counts on the request and response identifiers are the signature of the flood. Together the four captures give three detection features: the presence of the SDO identifiers, the command byte that tells read from write from abort, and the frame count that separates a single operation from a flood. The attack mainly changes the identifier order and the rate.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
1326	0x701	1	0x05	Heartbeat, operational
1141	0x601	8	0x40 0x00 0x60	SDO upload request
1141	0x581	8	0x4b 0x00 0x60	SDO upload response
2653	0x281	8	0xef 0xbe 0xbb	TPDO2, cyclic
13265	0x181	8	0x53 0x01 0xab	TPDO1, cyclic

Table 6.13.: Capture of the SDO flood on object 0x6000.

6.3.2. Detection

The SDO attacks were run against the blue node in turn, and each one produced alerts on the host SOC console. The first three are single careful transfers, and each raises one alert. The fourth is a load attack, and it raises a stream of alerts that change rule as the rate crosses the threshold.

Detection of the unauthorized read. The node detected an SDO read on victim node during normal operation, where no SDO traffic is expected. The rule R7c_SDO_ACCESS fired at the

warning level, shown in Table 6.14, because a single read in otherwise idle operation is the signal, since the baseline shows no SDO traffic.

Field	Value
Severity	WARNING
Rule	`R7c_SDO_ACCESS`
Suspect node	`0x01`
Suspect COB	`0x601`
Transfer	read, index `0x1017`, command `0x40`

Table 6.14.: SOC alert for the unauthorised SDO read.

Detection of the unauthorized write. The node detected an SDO write on node `0x01` during normal operation. The same rule `R7c_SDO_ACCESS` fired, but this time at the critical level, shown in Table 6.15. One rule covers both the read and the write case, and the severity comes from the command type. A write is graded critical because it changes a stored value in the object dictionary, while a read only observes one. An unauthorised write can alter the configuration or the process data of the victim, so it is a direct attack on the integrity of the node, which is why it is raised above the read.

Field	Value
Severity	CRITICAL
Rule	`R7c_SDO_ACCESS`
Suspect node	`0x01`
Suspect COB	`0x601`
Transfer	write, index `0x6001`, command `0x2B`

Table 6.15.: SOC alert for the unauthorised SDO write.

Detection of the refused write. This case is different from the first two, because the signal came from the response rather than from the request. The node saw an SDO abort sent by the victim, and the rule `R7b_SDO_ABORT` fired, shown in Table 6.16. The abort proves that an access was tried and rejected. The victim enforced the read only access right of the object, and the node turned that refusal into evidence of the attempt. The abort rule therefore catches the attack even when the write does not succeed, because the refusal itself is the trace.

Field	Value
Severity	CRITICAL
Rule	`R7b_SDO_ABORT`
Suspect node	`0x01`
Suspect COB	`0x581`
Transfer	abort, index `0x1017`, command `0x80`

Table 6.16.: SOC alert for the refused SDO write.

Detection of the flood. The flood capture shows the node changing its response as the rate grew, summarised in Table 6.17. At first each request raised the access rule `R7c_SDO_ACCESS` at the warning level, because each request is also an unsolicited read. Once the request

count passed the threshold of ten requests in one second, the node switched to the flood rule `R8_SDO_FLOOD` at the critical level. The flood rule does not only fire once. It reports the live count against the threshold as the rate climbs, so the operator sees both that a flood is in progress and how strong it is.

Phase	Rule	Severity
Below threshold	`R7c_SDO_ACCESS`	WARNING
Above threshold	`R8_SDO_FLOOD`	CRITICAL

Table 6.17.: SOC alert stream during the SDO flood.

Summary of the SDO defense. The node detected every SDO attack and matched the right rule to each one, with the severity graded by the operation. The unknown node rule `R7a_SDO_UNKNOWN_NODE` did not fire, because every attack targeted the whitelisted victim `0x01`. This category shows the value of the specification based approach, because the node does not need a heartbeat from the source to detect an SDO access, which is the property that catches the stealth case discussed in the design.

6.4. PDO Injection

The PDO case built two attacks, a single spoofed frame and a continuous flood, both on a genuine PDO identifier. This section shows the trace each one left on the bus and how the detector reacted to it.

6.4.1. Attack Signature

The PDO injection differs from the NMT and SDO attacks in one important way. The spoofed frames do not use a new identifier. They reuse the identifier of a real transmit PDO, `0x181` or `0x281`. In the grouped view this means the injected frame does not create a new row. It only raises the count of the existing row and replaces the displayed payload. For this reason the grouped view cannot show the injection as a separate frame, a limitation that is stated openly because it shapes how the evidence must be read.

Table 6.18 shows the single injection. The row for `0x181` still exists as in the baseline, so the identifier alone gives no signal. The signal is in the payload, which no longer follows the regular counting pattern of the genuine TPDO1. This is indirect proof. It shows that a frame with a foreign payload reached the identifier, but it cannot place the genuine and spoofed frames side by side. The heartbeat stays operational throughout.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
218	`0x701`	1	`0x05`	Heartbeat, operational
435	`0x281`	8	`0x64 0x00 0xc4`	TPDO2, cyclic
2174	`0x181`	8	`0x30 0x02 0xd4`	TPDO1, payload altered

Table 6.18.: Capture during the single PDO injection on `0x181`.

Table 6.19 shows the flood. The payload on `0x181` carries a clear crafted marker, and the count reaches `5434`, far above the other identifiers in the same window. This count is much higher than the genuine period would produce, which points to an elevated frame rate. As with the single injection, the grouped view cannot separate the two sources and can only show

the combined count and the last payload. To prove the injection as two distinct frames within one period, a time ordered capture is required, which is exactly the linear stream the detector reads. The attack therefore changes the rate, which is the feature the detection relies on.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
367	`0x701`	1	`0x05`	Heartbeat, operational
733	`0x281`	8	`0x64 0x00 0xee`	TPDO2, cyclic
5434	`0x181`	8	`0xff 0xff 0x13`	TPDO1, flooded with crafted data

Table 6.19.: Capture during the continuous PDO injection flood on `0x181`.

6.4.2. Detection

The PDO injection attacks were run against the blue node, and each one produced alerts on the host SOC console, shown in the tables below. The grouped capture could not show this attack as a separate frame, because the injected frame and the genuine frame share the identifier `0x181`. The node detects it through timing, by measuring the gap between consecutive frames on that identifier.

Detection of the single injected frame. The genuine TPDO1 runs at 100 milliseconds, so the node expects at least that gap between two frames. The injected frame arrived 44 milliseconds after the previous one, below the 50 millisecond limit, which is half the nominal period. A single producer cannot send two frames this close, so the second frame must be injected. The rule `R_PDO_INJECT` fired at the warning level, shown in Table 6.20. This capture is the proof that the timing approach works. The grouped view could not separate the two frames, but the node, reading the bus as a linear stream, measured the short gap and flagged the frame. The payload was not needed. The timing alone was enough.

Field	Value
Severity	WARNING
Rule	`R_PDO_INJECT`
Suspect node	`0x01`
Suspect COB	`0x181`
Measured gap	44 ms, below the 50 ms limit

Table 6.20.: SOC alert for the single PDO injection.

Detection of the continuous injection flood. In the flood the rule `R_PDO_INJECT` fired again and again, once for each injected frame, summarized in Table 6.21. The measured gaps were very short, between 8 and 12 milliseconds, far below both the 50 millisecond limit and the 100 millisecond genuine period. This is the same rule as the single injection, but here it fires continuously, because every injected frame arrives too soon after the one before it. The stream of alerts with very short gaps is the signature of a sustained injection.

Field	Value
Severity	WARNING
Rule	`R_PDO_INJECT`
Suspect node	`0x01`
Suspect COB	`0x181`
Measured gaps	8 ms to 12 ms, all below the 50 ms limit

Table 6.21.: SOC alert stream during the continuous PDO injection flood.

Detection of the flood together with a heartbeat loss. This capture is a finding in its own right, because it shows a side effect of the flood, summarized in Table 6.22. The stream of `R_PDO_INJECT` warnings continued as before. In the middle of it a critical alert appeared from a different category. The rule `R6_HEARTBEAT_LOST` fired and reported that the heartbeat from node `0x01` had been missing for 2014 milliseconds. The reason is that the flood filled the bus with high priority frames on `0x181`. The genuine heartbeat runs on a lower priority identifier, so it lost arbitration and could not get onto the bus in time. The 2000 millisecond timeout then elapsed, and the periodic check raised the loss alert. This shows that the PDO flood does more than inject false data. It also starves lower priority traffic, including the health signal of the node it targets. The node detected both effects at once, the injection through the timing rule and the heartbeat loss through the timeout rule, which gives the operator a fuller picture of the damage.

Event	Rule	Severity
Injected frames	`R_PDO_INJECT`	WARNING
Heartbeat gone	`R6_HEARTBEAT_LOST`	CRITICAL

Table 6.22.: SOC alert stream during the PDO flood with a heartbeat loss.

Summary of the PDO defense. The node caught every PDO injection by timing, because the injected frame arrives too soon after the genuine one, and it did so without needing the payload. The source identity rule `R_PDO_UNKNOWN_NODE` did not fire, because the attacker spoofed the identifier of the real victim `0x01` rather than a foreign node ID. The third capture adds a finding that the design predicted in part, that a high priority flood starves lower priority traffic, which the node reported through a separate rule.

6.5. Heartbeat Manipulation

The heartbeat case built two attacks, a spoof of the victim heartbeat and a ghost node that no real node owns. This section shows the trace each one left on the bus and how the detector reacted to it.

6.5.1. Attack Signature

The two heartbeat attacks differ in how visible they are in the grouped view, and this difference is itself a result. The spoof reuses the victim identifier `0x701` and folds into the existing row, while the ghost node uses an identifier that no real node owns and creates a new row.

In the spoof, shown in Table 6.23, the row for `0x701` exists exactly as in the baseline, and it still reports the operational state. The grouped view cannot separate a genuine heartbeat from a spoofed one when both share the identifier and claim the same state, so the only weak

trace is a raised count. This is the hard case, and its proof requires a time ordered capture in which two heartbeat frames appear inside one heartbeat period, which is impossible for a single producer.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
228	0x701	1	0x05	Heartbeat, operational
410	0x281	8	0x64 0x00 0xfd	TPDO2, cyclic
2051	0x181	8	0x50 0x03 0xf4	TPDO1, cyclic

Table 6.23.: Capture during the spoofed heartbeat on the victim identifier 0x701.

The ghost node in Table 6.24 is directly observable. The identifier 0x77E appears as a new row that the baseline never shows. It maps to node ID 0x7E for which no real node exists on the testbed, and it claims the operational state. A heartbeat consumer that keeps a node list would add this node and treat it as alive. The count of 12 shows the ghost was a repeating signal rather than a stray frame, which is what makes it look like a real periodic producer.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
12	0x77e	1	0x05	Ghost heartbeat, claims operational
115	0x701	1	0x05	Heartbeat, operational
229	0x281	8	0x64 0x00 0x64	TPDO2, cyclic
1145	0x181	8	0xd5 0x03 0xf5	TPDO1, cyclic

Table 6.24.: Capture during the ghost node heartbeat on 0x77E.

6.5.2. Detection

The heartbeat attacks were run against the blue node, and each one produced alerts on the host SOC console, shown in the tables below. These are the two cases that the attack chapter treated as the hard case and the easy case. The spoof reuses the identifier of a real node, so the node catches it by timing. The ghost node uses an identifier that no real node owns, so the node catches it directly by the identifier.

Detection of the spoofed victim heartbeat. The genuine heartbeat runs at 1000 milliseconds, so the node expects a long gap between two heartbeats. The minimum interval below which a second source must be present is set to 200 milliseconds. In the capture the node measured gaps of 70 to 73 milliseconds, far below both the minimum and the genuine period, summarized in Table 6.25. A single producer holds one fixed period and cannot send two heartbeats this close together, so the second must come from a second source, which is the spoof. The rule R6b_HEARTBEAT_SPOOF fired at the critical level for each pair. This is the key result for the hard case. The spoof copied the identifier of a real node, so it could not be caught by the identifier, but the node caught it by the timing of the arrivals, which does not depend on the content of the frame. Even though the spoofed heartbeat claimed the same operational state as the genuine one.

Field	Value
Severity	CRITICAL
Rule	`R6b_HEARTBEAT_SPOOF`
Suspect node	`0x01`
Suspect COB	`0x701`
Measured gaps	70 ms to 73 ms, below the 200 ms minimum

Table 6.25.: SOC alert stream during the spoofed victim heartbeat.

Detection of the ghost node. The node received a heartbeat on `0x77E`, which maps to node ID `0x7E`. That node is not in the whitelist, because no real node with that identifier exists on the testbed. The rule `R6a_GHOST_NODE` fired at the critical level, listed in Table 6.26. This is the easy case. The node needed no timing measurement or content check. The identifier itself was the signal, because a heartbeat on a node ID that the network does not contain is an announced node that is not really there. A heartbeat consumer that built a node list would have added this false node and treated it as alive, but the blue node flagged it as soon as it appeared.

Field	Value
Severity	CRITICAL
Rule	`R6a_GHOST_NODE`
Suspect node	`0x7E`
Suspect COB	`0x77E`
Claimed state	`0x05`, operational

Table 6.26.: SOC alert stream during the ghost node heartbeat.

Summary of the heartbeat defense. The spoof on the real identifier was caught by timing, because two heartbeats arrived far closer than a single producer would allow. The ghost node was caught by the identifier, because its node ID was not in the whitelist. Both were raised as critical. The false state rule `R6c_HEARTBEAT_STATE` did not fire, because both attacks claimed the valid operational state. The loss rule `R6_HEARTBEAT_LOST` did not fire in these runs, although it was shown firing in the PDO flood capture, where the heartbeat was starved off the bus. Together the two rules cover both the hard and the easy case of heartbeat manipulation.

6.6. EMCY Injection

The EMCY case built three attacks, a single generic fault, a single overvoltage fault, and a flood. This section shows the trace each one left on the bus and how the detector reacted to it.

6.6.1. Attack Signature

The EMCY attack is the clearest of the application layer attacks in the grouped view, because the identifier `0x081` never appears in normal operation. Any frame on it is therefore a new row that did not exist before.

The captured frame in Table 6.27 claims a communication error and sets the matching error register bit. The conflict is clear, because the heartbeat still reports operational and both PDOs keep running. A node that reports a communication error while it produces correct

cyclic traffic and a healthy heartbeat is in a contradictory state, and this mismatch is the signature the detector checks.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
217	0x701	1	0x05	Heartbeat, operational
435	0x281	8	0x64 0x00 0x03	TPDO2, cyclic
2175	0x181	8	0x6e 0x03 0x12	TPDO1, cyclic
1	0x081	8	0x00 0x81 0x10	Fake EMCY, error code 0x8100

Table 6.27.: Capture during the single fake EMCY injection on 0x081.

The overvoltage variant, in Table 6.28, claims a voltage fault. This is more specific than the generic fault, because it claims a concrete physical condition that an operator or supervising node might act on, for example by shutting the device down. The rest of the capture again contradicts the claim, because the node continues normal operation while reporting the fault.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
305	0x701	1	0x05	Heartbeat, operational
611	0x281	8	0x64 0x00 0xb3	TPDO2, cyclic
3053	0x181	8	0x58 0x03 0x80	TPDO1, cyclic
2	0x081	8	0x10 0x32 0x04	Fake EMCY, error code 0x3210

Table 6.28.: Capture during the single fake EMCY overvoltage injection on 0x081.

Table 6.29 records the flood. It claims a manufacturer specific fault, and the feature that separates it from the single injection is the count of 122. The victim never produces a stream of emergencies during normal operation, so a count this high is itself abnormal regardless of the error code. A flood of emergencies consumes bandwidth and buries any genuine fault in noise. In every variant the claim conflicts with the rest of the capture, which is the central feature for detection.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
357	0x701	1	0x05	Heartbeat, operational
715	0x281	8	0x64 0x00 0x1b	TPDO2, cyclic
3574	0x181	8	0xdd 0x01 0x89	TPDO1, cyclic
122	0x081	8	0x00 0x90 0x80	EMCY flood, error code 0x9000

Table 6.29.: Capture during the EMCY flood on 0x081.

6.6.2. Detection

The EMCY attacks were run against the blue node, and each one produced alerts on the host SOC console, shown in the tables below. The emergency identifier carries no traffic in the idle baseline, so every emergency frame in these runs is already abnormal. In practice the signature rule fired first on the single injections, because the red node marks its emergency frames with a fixed signature that the node checks directly.

The red node tags every emergency frame it sends with a fixed value in the manufacturer specific bytes. An EMCY frame leaves five bytes free after the error code and the error register, and the red node always writes the same recognisable value there. The rule R5_SIGNATURE_MATCH looks for that value and fires when it sees it. This tag is a testbed aid that lets the detector attribute a frame to the red node during the experiments. A real attacker would not include

it, so the signature rule is not the general detection mechanism. The general mechanism is the contradiction between the claimed fault and the healthy state that the heartbeat and the process data still report.

Detection of the single fake emergency. The node received the emergency on `0x081` and checked the manufacturer bytes against the known red node signature. The bytes matched, so the rule `R5_SIGNATURE_MATCH` fired at the critical level, shown in Table 6.30. This rule is the sharpest, because it does not only show that the emergency is suspect, it identifies the source as the red node directly. The signature rule fired here rather than the error code rule, because the node checks the signature and returns on the first match. The single emergency was therefore detected at once, and the alert pointed straight at the attacker.

Field	Value
Severity	CRITICAL
Rule	`R5_SIGNATURE_MATCH`
Suspect node	`0x01`
Suspect COB	`0x081`
Signal	attacker signature in manufacturer bytes

Table 6.30.: SOC alert for the single fake EMCY injection.

Detection of the single fake overvoltage emergency. The attack claimed a different fault, an overvoltage rather than a generic error, but the node detected it the same way, shown in Table 6.31. Both frames came from the red node and carried the same signature, so the signature rule caught them both before the error code was examined. This shows an important property of the rule. It does not depend on what fault the attacker claims. Whatever error code the red node sets, the signature stays the same, so the node catches the frame regardless of the claimed fault.

Field	Value
Severity	CRITICAL
Rule	`R5_SIGNATURE_MATCH`
Suspect node	`0x01`
Suspect COB	`0x081`
Signal	attacker signature in manufacturer bytes

Table 6.31.: SOC alert for the single fake EMCY overvoltage injection.

Detection of the flood. The flood capture shows two rules firing together, summarised in Table 6.32. Some frames raised the signature rule `R5_SIGNATURE_MATCH`, because they carried the red node signature. Others raised the flood rule `R2_EMCY_FLOOD`, which reported the rising count against the threshold of three emergencies in one second. Both rules ran at the critical level. The reason two rules appear is the order of the checks. The flood rule is a rate check, while the signature rule is a content check on the manufacturer bytes. Together they show both facts at once, that the emergencies came from the red node and that they arrived at a flood rate. The source is named by the signature, and the load is measured by the flood count.

Event	Rule	Severity
Signature match	R5_SIGNATURE_MATCH	CRITICAL
Rate over threshold	R2_EMCY_FLOOD	CRITICAL

Table 6.32.: SOC alert stream during the EMCY flood.

Summary of the EMCY defense. Both single injections raised the signature rule R5_SIGNATURE_MATCH, which identified the red node directly and did not depend on the claimed fault. The flood raised the flood rule R2_EMCY_FLOOD alongside the signature rule. The unknown node rule R1_UNKNOWN_NODE_EMCY did not fire, because the attacker used the victim node ID 0x01. The timing rule R3_EMCY_TIMING and the error code rule R4_BAD_ERROR_CODE did not fire, because the signature and flood rules caught the frames first.

6.7. LSS Abuse

The LSS case built two attacks, one that sets a new node ID and one that sets a new bitrate. This section reports what each one produced on the bus and the alert it raised.

6.7.1. Attack Signature

The LSS attack is visible in the grouped view for the same reason as the EMCY attack. The LSS master identifier 0x7E5 never appears in normal operation, because LSS runs only during a configuration sequence. Any frame on it is therefore a new row that the baseline never shows, so the first level of success is directly observable. The victim node does not implement the slave side of LSS, so it never applies the requested node ID or bitrate and never replies on the LSS slave identifier 0x7E4. This is by design. The testbed only needs to show that the command is detectable on the bus, not that the victim carries it out, because the detector watches the bus regardless of how the victim reacts. The captures therefore show the attack frames and their detection, while the victim keeps its original node ID, bitrate, and traffic throughout.

A stronger attacker would send a complete and valid sequence. The LSS master must first switch the target into the LSS configuration state, either with a switch mode global frame or, to address one node, with the selective switch frames that carry its LSS identity. This does not weaken the detection. Every such frame is an LSS master frame on 0x7E5, and R_LSS_ABUSE flags any LSS master frame during operation, so a more complete sequence produces more alerts and an earlier one, not fewer.

In Table 6.33 the command requests a new node ID. No reply appears on the LSS slave identifier 0x7E4, and the victim keeps its original heartbeat and process data, so only the first level of success was reached and the identity change had not taken effect in this window. The presence of the LSS command during operation is still a clear and abnormal event.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
2	0x7e5	8	0x11 0x10 0x00	LSS configure node ID
72	0x701	1	0x05	Heartbeat, operational
144	0x281	8	0x64 0x00 0xad	TPDO2, cyclic
716	0x181	8	0xc5 0x03 0x61	TPDO1, cyclic

Table 6.33.: Capture during the LSS abuse that sets a new node ID.

In the bitrate variant, recorded in Table 6.34, the command requests a new bus speed. As with the node ID case, no reply appears on 0x7E4 and the victim stays on the bus, so only

the first level was reached. This is the safer outcome, because a bitrate change that took effect would make the victim speak at a speed the rest of the network does not use and would remove it from the capture entirely.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
4	0x7e5	8	0x13 0x00 0x02	LSS configure bitrate
164	0x701	1	0x05	Heartbeat, operational
327	0x281	8	0x64 0x00 0x64	TPDO2, cyclic
1633	0x181	8	0xd6 0x03 0xf6	TPDO1, cyclic

Table 6.34.: Capture during the LSS abuse that sets a new bitrate.

6.7.2. Detection

The LSS attacks were run against the blue node, and each one produced alerts on the host SOC console, shown in the tables below. Each attack produced two alerts, not one, because the LSS protocol needs two steps. The attacker must first switch the slave into configuration mode and then send the actual configuration command. The node detected both steps. The single rule R_LSS_ABUSE fired on every LSS master frame, named the service, and reported each event at the critical level.

Detection of the LSS set node ID abuse. The node raised two alerts, both with the rule R_LSS_ABUSE at the critical level, shown in Table 6.35. The first named the command as switch mode global, the frame that moves the slave from its normal waiting state into configuration mode, where it will accept setting changes. The second named the command as configure node ID, the frame that sets the new node ID. Each was marked as a commissioning service abused, because LSS belongs to commissioning and must not run during operation. The node detected the attack at the first step and again at the second, so the attack was flagged before the node ID change could take any effect.

Step	Command	Specifier
1	switch mode global	0x04
2	configure Node-ID	0x11

Table 6.35.: SOC alerts for the LSS set node ID abuse.

Detection of the LSS set bitrate abuse. The node again raised two alerts, both with the rule R_LSS_ABUSE at the critical level, shown in Table 6.36. The first was the same switch mode global step as before. The second named the command as configure bitrate, the frame that would change the bus speed of the slave. This case is the most damaging if it succeeds. A bitrate change that took effect would make the victim speak at a speed the rest of the network does not use, and it would vanish from the bus entirely. The node flagged the configure bitrate frame the moment it appeared, before the change could take effect, so the attack was visible at the point of the command rather than only as a later loss of the node. The switch mode global frame gave an even earlier warning.

Step	Command	Specifier
1	switch mode global	0x04
2	configure bitrate	0x13

Table 6.36.: SOC alerts for the LSS set bitrate abuse.

Summary of the LSS defense. The node detected both LSS attacks and caught each one twice, once at the switch into configuration mode and once at the configuration command. The presence of any LSS master frame during operation was the signal, and the node did not need a victim reaction to detect it. The two step structure gave the node two chances to detect each attack, and it caught both steps every time. This is the cleanest and most reliable detection of all the categories, because the LSS identifier carries no traffic at all in a running network, so any frame on it is abnormal by definition.

6.8. Bus-Level Denial of Service

The denial of service case built one attack, a high priority flood on the lowest identifier that targets the shared medium rather than one service. This section reports what it produced on the bus and the alert it raised.

6.8.1. Attack Signature

The bus level denial of service is the most visible attack in the capture and also the least subtle. It attacks the shared medium with a flood of frames carrying the lowest identifier 0x000, which always wins arbitration. Table 6.37 shows the result.

Count	CAN-ID	Len	Data (bytes 0,1,2)	Meaning
82	0x701	1	0x05	Heartbeat, operational
164	0x281	8	0x64 0x00 0xb4	TPDO2, cyclic
817	0x181	8	0x64 0x00 0x84	TPDO1, cyclic
187896	0x000	8	0x00 0x00 0x00	Flood frame, highest priority

Table 6.37.: Capture during the high priority bus saturation flood on 0x000.

The flood identifier reaches a count of 187896 in the capture window, more than two hundred times the most active genuine identifier. This imbalance is the proof of timing damage. The genuine counts are far below what their fixed periods should produce, which means the window captured far fewer genuine cycles than normal, because the flood occupied the bus. The genuine frames did not stop completely, but they were pushed into the gaps the flood left, and their effective rate dropped. A consumer that expects a TPDO1 every 100 ms would see frames arrive late or not at all while the flood ran, which is the loss and delay the success criterion calls for.

This attack is different from every other in this work. The other attacks send valid frames on service identifiers and abuse the meaning of those frames, so they work inside the protocol. The flood ignores the protocol. It sends raw high priority frames whose only purpose is to win arbitration and hold the bus, so it cannot be detected by checking the meaning of a CANopen service. It is detected by the load itself.

6.8.2. Detection

This attack behaved differently from all the others. The other attacks each produced one clear rule for their own category. The flood, because it attacks the shared medium rather than one service, disturbed many identifiers at once and triggered several rules from several categories in the same moment. Table 6.38 summarises the mixed reaction observed when the flood was sent on `0x000`.

Triggered rule	Category	Severity
`R_NMT_BAD_CS`	NMT abuse	WARNING
`R_PDO_INJECT`	PDO injection	WARNING
`R6b_HEARTBEAT_SPOOF`	Heartbeat manipulation	CRITICAL

Table 6.38.: SOC alert stream during the high priority bus saturation flood.

The flood frames on `0x000` were read by the NMT check, and because the payload matched no valid NMT command, `R_NMT_BAD_CS` fired repeatedly. At the same time the flood compressed the timing of the genuine traffic. The process data on `0x181` arrived in bursts, so the gaps fell far below the period and `R_PDO_INJECT` fired, even though no real injection took place. The flood also compressed the heartbeat timing, so `R6b_HEARTBEAT_SPOOF` fired at the critical level on `0x701`, and again on `0x281` below its own limit.

This mixed reaction is itself the signature of the flood. No other attack produced alerts across this many categories at once. In one sense the firing of the PDO and heartbeat rules is a false attribution, because those rules fired on traffic that was disturbed rather than injected. In another sense it is a correct and useful signal, because the simultaneous firing of rules from many categories, with very high counts and very short gaps, is a pattern that only a bus level flood produces. An operator who sees NMT and PDO and heartbeat alerts erupt together at high rate can read that pattern as a denial of service on the medium.

This capture also shows the limit that the detection logic stated in advance. The detector reported the flood clearly, but it could not prevent it. The arbitration damage was already done at the data link layer, where the high priority frames on `0x000` won the bus and pushed the genuine traffic aside. From the listen-only position the detector can report the flood but cannot remove the flood frames or restore the timing. This attack is the loudest on the bus and the least preventable, and it marks the boundary of what a passive specification based detector can do.

6.9. RQ1: Distinguishability of Attack Signatures

RQ1 asks whether attacks across the seven studied CANopen service categories produce signatures that can be told apart from the baseline and from each other. The per-category results above support this. Each category changed at least one of the four baseline features, and the signatures differ by category. The frame length is the one baseline feature that no attack changes, because every attack sends well formed frames of the expected length, so it does not appear among the changed features. Table 6.39 draws the results together.

Two categories carry a caveat. The PDO injection and the spoof on the victim heartbeat reuse an identifier that is already present, so they do not create a new row in the grouped view, and their proof needs the time ordered stream that the detector reads rather than the grouped capture. This limitation is reported openly, because it shapes how the signature must be confirmed. With that confirmation, the seven studied categories each produce a distinguishable and repeatable signature, which supports RQ1.

Category	Changed baseline features	Primary evidence
NMT	order, rate, data pattern	heartbeat state byte flips
SDO	order, rate	0x601/0x581 appear; command byte
PDO injection	rate, data pattern	short inter-arrival gap on 0x181
Heartbeat	order or data pattern	new heartbeat ID, or short gap on 0x701
EMCY	order	0x081 appears; fault vs. healthy state
LSS	order	0x7E5 appears during operation
Bus DoS	rate, order (whole bus)	count on 0x000 dwarfs all genuine IDs

Table 6.39.: Attack signatures by category, expressed as changes to the four baseline features and the primary evidence.

6.10. RQ2: Sources of the Detection Rules

RQ2 asks which detection rules for the attack categories can be taken from the CANopen specification alone, and which rules also need observed runtime behavior. The per-category results above answer this from the data already collected. Most rules rest on a behavior that the specification fixes and fire on a frame that departs from it. Some rules rest on a frame that is valid at the transport layer, and fire on its observed timing or rate.

The derivation follows one principle. A specification fixes what a compliant device may send on which identifier and in which state. Any frame that breaks one of these fixed rules is a candidate for an alert. Where a frame is valid but its timing or rate is not expected, the rule uses that observed runtime property instead. The detector does not learn a model from traffic. It encodes the rules the standard states, together with the expected runtime behavior, and reports the frames that violate them.

Table 6.40 lists each rule against the basis it rests on, the source that defines it, and its rule class. The rule class records which of the two rule sets a rule belongs to. A class A rule is a transport layer rule that judges a frame by its identifier and its rate, and a class B rule is a CANopen aware rule that judges a frame by what it means in the protocol. The transport layer rules also run when the CANopen aware rules are active, so a class A rule is not limited to Mode A, but it is the class of rule that Mode A alone can already provide.

Rule	Basis	Rule class
R_NMT_CONTROL	NMT control protocol, COB-ID 0x000	B (CANopen-aware)
R_NMT_BAD_CS	defined NMT command specifier set	B (CANopen-aware)
R7c_SDO_ACCESS	SDO client/server identifiers, transfer model	B (CANopen-aware)
R7b_SDO_ABORT	SDO abort transfer service	B (CANopen-aware)
R8_SDO_FLOOD	SDO request/response model	A (transport)
R_PDO_INJECT	PDO transmission type and event timing	A (transport)
R_PDO_UNKNOWN_NODE	default PDO identifier allocation	B (CANopen-aware)
R6a_GHOST_NODE	heartbeat producer identifier allocation	B (CANopen-aware)
R6b_HEARTBEAT_SPOOF	heartbeat producer time, single producer	B (CANopen-aware)
R6c_HEARTBEAT_STATE	defined NMT state set in the heartbeat	B (CANopen-aware)
R1-R5 (EMCY)	EMCY object and error code table	B (CANopen-aware)
R_LSS_ABUSE	LSS master identifier 0x7E5	B (CANopen-aware)
R_BUS_FLOOD_ID	priority arbitration on the shared medium	A (transport)

Table 6.40.: Detection rules and its rule class.

The NMT rules rest on the NMT control protocol. The standard fixes COB-ID `0x000` for node control and lists the valid command specifiers. So a frame on `0x000` is a control command, and a specifier outside the list is malformed.

The SDO rules rest on the transfer model and its fixed identifiers. The baseline shows no SDO traffic, so an unsolicited transfer, an abort, or a burst of requests each break the expected exchange.

The PDO rules rest on the transmission type and the default identifiers, which the specification fixes, together with the observed rate on the bus. The specification defines the expected period, but the rule needs the observed rate to decide that a frame arrives too early. A frame that arrives earlier than its period cannot come from the single genuine producer, and a PDO with a foreign node ID marks a foreign source.

The heartbeat rules rest on the producer model, the producer time, and the defined states. A heartbeat from an unknown node, two heartbeats too close together, or an invalid state byte each break a fixed property.

The EMCY rules rest on the emergency object and its error code table. The LSS rule rests on the LSS master identifier, which appears only at commissioning, so any such frame during operation is abnormal.

The bus load rules rest on the priority arbitration of the shared medium. They judge a frame by its rate alone, so they are transport layer rules and stay active in both modes, like the other rate based rules.

The Mode A and Mode B comparison gives the practical answer to RQ2. Mode A uses only the transport layer rules, which judge a frame by its identifier and its rate. Mode B adds the CANopen aware rules, which judge a frame by what it means in the protocol.

Two attacks are the sharpest case, because they reuse an identifier that is already valid on the bus and therefore leave no transport layer trace at all. The SDO stealth access reuses the client identifier `0x601`, which is normal during operation, and the heartbeat spoof reuses the victim identifier `0x701`. In both cases the identifier is genuine, so nothing about how the frame is sent looks wrong. Only a rule that reads the CANopen meaning can flag them, the unauthorized object access in the first case and the two heartbeats too close together in the second.

The other CANopen aware rules, such as the NMT, EMCY, and LSS rules, also belong to Mode B, because reading the frame still needs CANopen knowledge, for example to know that `0x7E5` is the LSS master identifier. But the attacks they catch are easier, because they use an identifier that the baseline never shows, so their mere appearance is already a signal. These attacks are caught by a CANopen aware rule, yet they would also disturb a purely identifier based view.

This is the two-part answer RQ2 asks about. Some rules follow from the CANopen specification and catch attacks that Mode A cannot see at all, and the two reused-identifier cases are the clearest proof of this. Others rest on observed runtime behavior such as rate, and these can already act at the transport layer, which is why the PDO injection, the SDO flood, and the bus load rules stay active in both modes.

7. Summary and Outlook

7.1. Summary

This thesis set out to close a precise gap. CANopen is widely deployed in industry, it is built for open modular growth, and its services are not secured at the CANopen level, yet no published specification based intrusion detection system existed for the CANopen service layer on microcontroller hardware. The work built and evaluated such a system on a three node ESP32 testbed under a red team and blue team structure.

The work was organized in four phases. A baseline phase recorded the normal traffic and established the four stable features of rate, length, data pattern, and identifier order. It confirmed that the passive detector does not disturb the bus. An attack phase ran a systematic attack tool across seven CANopen service categories and recorded the signature each attack. A rule derivation phase encoded the expected behavior of CiA 301 and CiA 305 into a small set of detection rules. An evaluation phase ran every attack against the rules in two modes, a transport layer mode (Mode A) that uses the kind of information a CAN IDS works with and a full CANopen aware mode (Mode B).

The results support the mentioned research questions.

For RQ1, the seven studied attack categories each produced a distinguishable and repeatable signature in at least one baseline feature. Two cases, the PDO injection and the heartbeat spoof, reuse an existing identifier and need the detector timeline to confirm.

For RQ2, the detector raised the correct alert for every category. Most rules map to a behavior fixed by the specification. A few rules also need observed runtime behavior, such as message rate, because the frame itself is compliant. The two mode design showed that the CANopen aware rules catch cases the transport layer rules cannot, in particular the stealthy SDO access and the heartbeat spoof. The PDO injection was caught at the transport layer by its rate. This is the central argument of the work. A CAN IDS checks how frames are sent, while a CANopen IDS must also understand what frames mean and how often they arrive, and only the latter catches these attacks.

The work also produced findings beyond the research questions. The SDO stealth access, which manipulates the object dictionary without any heartbeat announcement, is undetectable by any heartbeat based approach and is caught only by the SDO rules. The PDO injection flood was shown to starve the lower priority heartbeat off the bus. The detector reported both the injection and the resulting heartbeat loss at once. The bus level denial of service was shown to be the most visible attack and the least preventable, because the arbitration damage happens at the data link layer below the reach of any specification based logic.

7.2. Limitations

The work has clear limitations that bound its claims. The testbed uses one victim node and a single whitelisted node ID, so the detector was exercised against attacks aimed at one known target rather than against a large commissioned network. Several rules, such as the unknown node rules, did not fire in these runs because the attacks used the victim node ID, so their behavior against a foreign node ID is argued from the logic rather than measured. The

SYSWORXX grouped view cannot separate an injected frame from a genuine one on a shared identifier, which is why the injection and spoof cases rely on the detector timeline for proof. The detector runs from a listen-only position, so it can detect a bus level flood but cannot prevent it. Each limitation bounds a specific claim rather than undermining the result as a whole.

7.3. Outlook

Several directions follow from this work. The most direct is to scale the testbed to a multi-node network with several commissioned node IDs, which would exercise the unknown node rules against real foreign traffic and would test the whitelist under a more realistic commissioning. A second direction is to move the detector from detection toward active prevention from a node that is not listen-only, which would let it answer a bus level flood rather than only report it, at the cost of no longer being a purely passive monitor. A third direction is to broaden the attack tool with further attacks that follow the CANopen rules at the frame level but abuse the meaning of a service, since these semantic attacks are the cases where a CANopen aware detector should most clearly outperform a transport layer one. A final direction is to test the rule set against a commercial CANopen stack rather than the custom one used here, which would confirm that the rule triggers correspond to genuine protocol violations across implementations and not only to the behavior of the testbed stack.

List of Tables

List of Figures

A. Source Code and Data Availability

The full source code of the testbed is published in a public repository to support transparency and reproducibility:

`https://github.com/sandjberd/CANOpen_IDS_FH_Joanneum_IMS`

The repository is organised into four parts that match the roles described in Chapter 5. The directory `red_team_node` holds the ESP-IDF firmware of the attacker, `victim_node` holds the custom CANopen slave stack, and `blue_team_node` holds the detection engine that runs the TWAI controller in listen-only mode. The directory `host_listener` holds the Python host tools, which are the command and control console, the SOC listener that records the detection events, and a software victim used during development. Each firmware project keeps its parameters in a `config.h` file, so the node identifiers, the PDO periods, and the detection thresholds are set in one place.

A.1. Passive Bus Observation

The blue node installs the TWAI controller in listen-only mode, so it receives every frame but never acknowledges and never transmits. Listing A.1 shows the initialisation. This is the code that guarantees the property claimed in Section 4.2, namely that the detector cannot alter the traffic it measures.

```
esp_err_t can_monitor_init(void)
{
    twai_general_config_t g_config = TWAI_GENERAL_CONFIG_DEFAULT(
        CAN_TX_GPIO, CAN_RX_GPIO, TWAI_MODE_LISTEN_ONLY);
    g_config.rx_queue_len = 64;

    twai_timing_config_t t_config = TWAI_TIMING_CONFIG_500KBITS();
    twai_filter_config_t f_config = TWAI_FILTER_CONFIG_ACCEPT_ALL();

    esp_err_t err = twai_driver_install(&g_config, &t_config, &f_config);
    if (err != ESP_OK) return err;
    err = twai_start();
    if (err != ESP_OK) return err;

    ESP_LOGI(TAG, "TWAI sniffer running (listen-only) at %d kbps",
             CAN_BITRATE_KBPS);
    return ESP_OK;
}
```

Listing A.1: Blue node TWAI initialisation in listen-only mode.

A.2. A Specification-Derived Rule

Listing A.2 shows the NMT rule. It maps directly to CiA 301. The rule decodes the command specifier against the defined set, raises a malformed-frame alert for any specifier outside that

set, and grades the severity from the command itself, so that a stop, a reset node, or a reset communication is raised as critical. No runtime learning is involved. This illustrates the specification-derived side of RQ2.

```
bool ids_check_nmt(const can_frame_t *f, ids_alert_t *out_alert)
{
    char desc[128];
    if (f->cob_id != NMT_COB) return false;
    if (f->dlc < 2) return false;

    uint8_t cs     = f->data[0];
    uint8_t target = f->data[1];

    const char *name = "unknown";
    bool known = true;
    switch (cs) {
        case 0x01: name = "start";        break;
        case 0x02: name = "stop";         break;
        case 0x80: name = "enter pre-op"; break;
        case 0x81: name = "reset node";   break;
        case 0x82: name = "reset comm";   break;
        default:   known = false;         break;
    }

    if (!known) {
        snprintf(desc, sizeof(desc),
            "NMT frame with unknown command specifier 0x%02X", cs);
        fill_alert(out_alert, "R_NMT_BAD_CS", desc,
                   ALERT_WARNING, target, NMT_COB);
        return true;
    }

    alert_severity_t sev = (cs == 0x02 || cs == 0x81 || cs == 0x82)
                         ? ALERT_CRITICAL : ALERT_WARNING;
    snprintf(desc, sizeof(desc),
        "NMT '%s' (cs=0x%02X) for target 0x%02X observed", name, cs, target);
    fill_alert(out_alert, "R_NMT_CONTROL", desc, sev, target, NMT_COB);
    return true;
}
```

Listing A.2: NMT detection rule, derived from the CiA 301 command specifier set.

A.3. A Runtime-Derived Rule

Listing A.3 shows the timing core of the PDO injection rule. The transmit PDO has a fixed period, so a frame that arrives earlier than half that period cannot come from the single genuine producer. This rule cannot rest on a clause alone, because the injected frame is conformant at the transport layer. It is the runtime-derived side of RQ2, and it is the check that catches the injection the grouped capture cannot show.

```
uint64_t now = f->timestamp_us;
uint64_t *last = is_tpdo1 ? &n->last_tpdo1_us : &n->last_tpdo2_us;
uint32_t period_ms  = is_tpdo1 ? TPDO1_PERIOD_MS : TPDO2_PERIOD_MS;
uint32_t min_gap_ms = (period_ms * PDO_EARLY_FACTOR_PCT) / 100;

if (*last != 0) {
    uint64_t delta_ms = (now - *last) / 1000;
    if (delta_ms < min_gap_ms) {
        snprintf(desc, sizeof(desc),
```

```
            "TPDO%d on Node 0x%02X arrived %llu ms apart (< %lu ms) --
                injected",
            is_tpdo1 ? 1 : 2, node_id,
            (unsigned long long)delta_ms, (unsigned long)min_gap_ms);
        fill_alert(out_alert, "R_PDO_INJECT", desc,
                   ALERT_WARNING, node_id, (uint16_t)f->cob_id);
        *last = now;
        return true;
    }
}
```

Listing A.3: PDO injection rule, based on the observed inter-arrival time on one identifier.

A.4. Detection Thresholds

The thresholds used by the rules are held in one header, so every value in the evaluation is traceable to a single place. Listing A.4 shows the relevant part. These are the values behind the results in Chapter 6, for example the 50 ms PDO gap and the EMCY rate threshold of three.

```
#define EMCY_RATE_THRESHOLD        3     /* EMCY per second */
#define EMCY_MIN_INTERVAL_MS       50

#define HEARTBEAT_TIMEOUT_MS       2000
#define HEARTBEAT_MIN_INTERVAL_MS  200

#define SDO_RATE_THRESHOLD         10    /* SDO requests per second */

#define TPDO1_PERIOD_MS            100
#define TPDO2_PERIOD_MS            500
#define PDO_EARLY_FACTOR_PCT       50    /* flag frames < 50% of period */

#define BUSLOAD_ID_THRESHOLD       200   /* frames per COB-ID per second */
#define BUSLOAD_TOTAL_THRESHOLD    1500  /* total frames per second */
```

Listing A.4: Detection thresholds from `blue_team_node/main/config.h`.

A.5. Capture Data

The CAN capture logs recorded by the SYSWORXX module during the runs are the ground truth for the results in Chapter 6. They are available on request and, together with the SOC alert logs written by the host listener, allow each reported signature and alert to be traced back to the run that produced it.

B. Use of Artificial Intelligence Assistance Tools

In accordance with the study regulations, the use of Artificial Intelligence (AI) assistance tools during the creation of this Master's Thesis is disclosed transparently below.

AI assistance tools were not used to generate original text, passages, or sections of this thesis. All text was written first by the author. AI tools (ChatGPT) were used afterwards only to correct English grammar and to rephrase existing sentences for clarity, as the author is not a native English speaker. The AI-refined output was not copied directly into the thesis. Each suggestion was reviewed, and the final wording was written by the author. The original text parts, prior to refinement, have been saved and can be provided upon request by the supervisor.

For the software implementation, GitHub Copilot was used to assist with writing and debugging code in Python and C, including troubleshooting and fixing errors. Given the author's limited prior experience with hardware, this assistance was particularly helpful in resolving hardware-related implementation issues.

AI assistance tools were used to produce the 2D graphical renderings of the hardware. These renderings were created on the basis of photographs of the physical hardware taken by the author.

All AI-assisted work was reviewed, verified, and validated by the author, who takes full responsibility for the content of this thesis.

Sample Prompts

The following prompts are representative examples of how AI assistance tools were used. They illustrate the nature of the assistance and are not an exhaustive list.

Language Refinement

> "correct the grammar in the following paragraph without changing the context: [original paragraph]."
>
> "rephrase this sentence to sound not too clumsy and academical: [original sentence]."
>
> "check this sentence grammatically correct: [original sentence]."
>
> "provide different wordings for this word: [word]"

Software Implementation and Debugging

> "this esp library does not contain the function for repeatable connection, provide a function to connect automatically if an error on the bus appears."
>
> "what does this error code mean: [error message and code snippet]."
>
> "write a Python parser to parse a CAN message and persist it and extract frames by cob-id."

Graphical Renderings

> "based on this photograph of a CAN transceiver board, generate a clean 2D schematic representation of this component."

C. Bibliography

Aliwa, Emad et al. (2021). "Cyberattacks and Countermeasures for In-Vehicle Networks". In: *ACM Computing Surveys* 54.1. DOI: 10.1145/3431233 (cit. on p. 13).

Axelsson, Stefan (2000). *Intrusion Detection Systems: A Survey and Taxonomy*. Technical Report 99-15. Göteborg, Sweden: Department of Computer Engineering, Chalmers University of Technology (cit. on p. 12).

Bloom, Gedare (2021). "WeepingCAN: A Stealthy CAN Bus-Off Attack". In: *Workshop on Automotive and Autonomous Vehicle Security (AutoSec)*. DOI: 10.14722/autosec.2021.23002 (cit. on p. 11).

Boeding, Matthew, Michael Hempel, and Hamid Sharif (2025). "End-to-End Framework for Identifying Vulnerabilities of Operational Technology Protocols and Their Implementations in Industrial IoT". In: *Future Internet* 17.1. DOI: 10.3390/fi17010034 (cit. on p. 13).

Bozdal, Mehmet et al. (2020). "Evaluation of CAN Bus Security Challenges". In: *Sensors* 20.8. DOI: 10.3390/s20082364 (cit. on p. 9).

CAN in Automation (CiA) (2011). *CiA 301: CANopen Application Layer and Communication Profile*. Standard. Version 4.2.0. CAN in Automation e.V. (cit. on pp. 1–5, 20).

– (2013). *CiA 305: CANopen Layer Setting Services (LSS) and Protocols*. Standard. Version 3.0.0. CAN in Automation e.V. URL: https://www.can-cia.org/can-knowledge/cia-305-layer-setting-services-lss (cit. on pp. 2, 4, 5).

– (2019). *CiA 412: CANopen Device Profiles for Medical Devices*. Standard. CAN in Automation e.V. URL: https://www.can-cia.org/can-knowledge/cia-412-canopen-profiles-for-medical-devices (cit. on p. 9).

Cho, Kyong-Tak and Kang G. Shin (2016). "Error Handling of In-Vehicle Networks Makes Them Vulnerable". In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. DOI: 10.1145/2976749.2978302 (cit. on p. 11).

Conti, Mauro, Denis Donadel, and Federico Turrin (2021). "A Survey on Industrial Control System Testbeds and Datasets for Security Research". In: *IEEE Communications Surveys & Tutorials* 23.4 (cit. on pp. 7, 14).

Cui, Jie et al. (2023). "Lightweight Encryption and Authentication for Controller Area Network of Autonomous Vehicles". In: *IEEE Transactions on Vehicular Technology* 72.11 (cit. on p. 5).

Debar, Hervé, Marc Dacier, and Andreas Wespi (1999). "Towards a Taxonomy of Intrusion-Detection Systems". In: *Computer Networks* 31.8. DOI: 10.1016/S1389-1286(98)00017-6 (cit. on p. 12).

Dönmez, Tahsin C. M. (2021). "Anomaly Detection in Vehicular CAN Bus Using Message Identifier Sequences". In: *IEEE Access* 9. DOI: 10.1109/ACCESS.2021.3117038 (cit. on p. 29).

Dzung, Dacfey et al. (2005). "Security for Industrial Communication Systems". In: *Proceedings of the IEEE* 93.6 (cit. on p. 9).

Espressif Systems (2025a). *ESP-IDF Programming Guide: FreeRTOS (Overview)*. ESP-IDF v5.5.1. Accessed: 2026-06-06. URL: https://docs.espressif.com/projects/esp-idf/en/release-v5.5/esp32/api-reference/system/freertos.html (cit. on p. 21).

– (2025b). *ESP-IDF Programming Guide: Two-Wire Automotive Interface (TWAI)*. ESP-IDF v5.5.1. Accessed: 2026-06-06. URL: https://docs.espressif.com/projects/esp-idf/en/release-v5.5/esp32/api-reference/peripherals/twai.html (cit. on p. 21).

European Parliament and Council (2022). *Directive (EU) 2022/2555 on Measures for a High Common Level of Cybersecurity Across the Union (NIS2 Directive)*. Official Journal of the European Union. L 333/80 (cit. on p. 10).

– (2023). *Regulation (EU) 2023/1230 on Machinery and Repealing Directive 2006/42/EC*. Official Journal of the European Union. OJ L 165, 29.6.2023, pp. 1–102. URL: https://eur-lex.europa.eu/eli/reg/2023/1230/oj (cit. on p. 10).

– (2024). *Regulation (EU) 2024/2847 on Horizontal Cybersecurity Requirements for Products with Digital Elements (Cyber Resilience Act)*. Official Journal of the European Union. OJ L, 2024/2847, 20.11.2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj (cit. on p. 10).

Groza, Bogdan and Pal-Stefan Murvay (2019). "Efficient Intrusion Detection With Bloom Filtering in Controller Area Networks". In: *IEEE Transactions on Information Forensics and Security* 14.4. DOI: 10.1109/TIFS.2018.2869351 (cit. on p. 29).

Hotellier, Estelle et al. (2024). "Standard Specification-Based Intrusion Detection for Hierarchical Industrial Control Systems". In: *Information Sciences* 659. DOI: 10.1016/j.ins.2024.120102 (cit. on pp. 12, 13).

Iehira, Kazuki, Hiroyuki Inoue, and Kenji Ishida (2018). "Spoofing Attack Using Bus-Off Attacks Against a Specific ECU of the CAN Bus". In: *2018 15th IEEE Annual Consumer Communications & Networking Conference (CCNC)*. DOI: 10.1109/CCNC.2018.8319180 (cit. on p. 11).

Im, H. and S. Lee (2024). "TinyML-Based Intrusion Detection System for In-Vehicle Network Using Convolutional Neural Network on Embedded Devices". In: *IEEE Embedded Systems Letters*. DOI: 10.1109/LES.2024.3475470 (cit. on p. 5).

International Electrotechnical Commission (2018). *IEC 62443: Security for Industrial Automation and Control Systems*. IEC Standard Series. Multi-part standard (cit. on p. 13).

International Organization for Standardization (2016). *ISO 11898-2: Road vehicles – Controller area network (CAN) – Part 2: High-speed medium access unit*. ISO. Geneva, Switzerland (cit. on p. 19).

Javed, Abbas et al. (2024). "Embedding Tree-Based Intrusion Detection System in Smart Thermostats for Enhanced IoT Security". In: *Sensors* 24.22. DOI: 10.3390/s24227320 (cit. on p. 13).

Jiang, Wenjun et al. (2021). "An Adaptive Intrusion Detection Algorithm for In-vehicle CAN Bus Based on Periodicity of Message". In: *Journal of Physics: Conference Series* (cit. on p. 29).

Johansson, Karl Henrik, Martin Törngren, and Lars Nielsen (2005). "Vehicle Applications of Controller Area Network". In: *Handbook of Networked and Embedded Control Systems* (cit. on p. 1).

Khraisat, Ansam et al. (2019). "Survey of intrusion detection systems: techniques, datasets and challenges". In: *Cybersecurity* 2.1. DOI: 10.1186/s42400-019-0038-7 (cit. on p. 12).

Larson, Ulf E., Dennis K. Nilsson, and Erland Jonsson (2008). "An Approach to Specification-Based Attack Detection for In-Vehicle Networks". In: *2008 IEEE Intelligent Vehicles Symposium*. IEEE (cit. on pp. 6, 7, 12, 13, 23).

Lee, Junseok et al. (2025). "ASIC Design for Real-Time CAN-Bus Intrusion Detection and Prevention System Using Random Forest". In: *IEEE Access* 13 (cit. on p. 5).

Lin, Hui et al. (2013). "Adapting Bro into SCADA: Building a Specification-Based Intrusion Detection System for the DNP3 Protocol". In: *Proceedings of the 8th Annual Cyber Security and Information Intelligence Research Workshop*. DOI: 10.1145/2459976.2459982 (cit. on pp. 12, 13).

Lotto, Alessandro et al. (2024). "A Survey and Comparative Analysis of Security Properties of CAN Authentication Protocols". In: *IEEE Communications Surveys & Tutorials* 26.4 (cit. on p. 5).

Makrakis, Georgios Michail et al. (2021). "Industrial and Critical Infrastructure Security: Technical Analysis of Real-Life Security Incidents". In: *IEEE Access* 9. DOI: 10.1109/ACCESS.2021.3133348 (cit. on pp. 9, 13).

Mirkovic, Jelena, Peter Reiher, et al. (2008). "Testing a Collaborative DDoS Defense in a Red Team/Blue Team Exercise". In: *IEEE Transactions on Computers* 57.8. DOI: 10.1109/TC.2008.42 (cit. on p. 14).

Murvay, Pal-Stefan and Bogdan Groza (2017). "DoS Attacks on Controller Area Networks by Fault Injections from the Software Layer". In: *Proceedings of the 12th International Conference on Availability, Reliability and Security (ARES)*. ACM (cit. on pp. 9, 27).

– (2018). "Security Shortcomings and Countermeasures for the SAE J1939 Commercial Vehicle Bus Protocol". In: *IEEE Transactions on Vehicular Technology* 67.5. DOI: 10.1109/TVT.2018.2795384 (cit. on p. 11).

Nankya, Mary, Robin Chataut, and Robert Akl (2023). "Securing Industrial Control Systems: Components, Cyber Threats, and Machine Learning-Driven Defense Strategies". In: *Sensors* 23.21 (cit. on p. 9).

Neto, Euclides Carlos Pinto et al. (2024). "CICIoV2024: Advancing Realistic IDS Approaches against DoS and Spoofing Attack in IoV CAN Bus". In: *Internet of Things* 26. DOI: 10.1016/j.iot.2024.101209 (cit. on p. 11).

Nweke, Livinus Obiora (2021). "A Survey of Specification-Based Intrusion Detection Techniques for Cyber-Physical Systems". In: *International Journal of Advanced Computer Science and Applications* 12.5. DOI: 10.14569/IJACSA.2021.0120506 (cit. on p. 12).

Oladimeji, Damilola et al. (2023). "CANAttack: Assessing Vulnerabilities within Controller Area Network". In: *Sensors* 23.19. DOI: 10.3390/s23198223 (cit. on pp. 11, 25).

Olufowobi, Habeeb et al. (2020). "SAIDuCANT: Specification-Based Automotive Intrusion Detection Using Controller Area Network (CAN) Timing". In: *IEEE Transactions on Vehicular Technology* 69.2. DOI: 10.1109/TVT.2019.2961344 (cit. on pp. 13, 29).

Pfeiffer, Olaf (2017). "Scalable CAN Security for CAN, CANopen and Other Protocols". In: *Proceedings of the iCC 2017 – CAN in Automation International Users and Manufacturers Group* (cit. on p. 23).

Popic, Srdjan et al. (2025). "Security Challenges and Mitigation Strategies for CAN-Based Protocols: A Comprehensive Survey". In: *24th International Symposium INFOTEH-JAHORINA (INFOTEH)*. IEEE (cit. on pp. 5, 11–13, 27).

Rajapaksha, Sampath et al. (2023). "AI-Based Intrusion Detection Systems for In-Vehicle Networks: A Survey". In: *ACM Computing Surveys* 55.11. DOI: 10.1145/3570954 (cit. on pp. 5, 12, 13).

Rajendran, Jeyavijayan, Vinayaka Jyothi, and Ramesh Karri (2011). "Blue Team Red Team Approach to Hardware Trust Assessment". In: *2011 IEEE 29th International Conference on Computer Design (ICCD)*. IEEE. DOI: 10.1109/ICCD.2011.6081410 (cit. on p. 14).

Rasheed, Amar et al. (2024). "Efficient Crypto Engine for Authenticated Encryption, Data Traceability, and Replay Attack Detection Over CAN Bus Network". In: *IEEE Transactions on Network Science and Engineering* (cit. on p. 5).

Reeves, Jason et al. (2011). "Lightweight Intrusion Detection for Resource-Constrained Embedded Control Systems". In: *IFIP International Conference on Critical Infrastructure Protection*. DOI: 10.1007/978-3-642-24864-1_3 (cit. on p. 13).

Scarfone, Karen and Peter Mell (2007). *Guide to Intrusion Detection and Prevention Systems (IDPS)*. NIST Special Publication Special Publication 800-94. Gaithersburg, MD, USA: Na-

tional Institute of Standards and Technology. DOI: 10.6028/NIST.SP.800-94 (cit. on p. 12).

Sekar, R. et al. (2002). "Specification-Based Anomaly Detection: A New Approach for Detecting Network Intrusions". In: *Proceedings of the 9th ACM Conference on Computer and Communications Security*. DOI: 10.1145/586110.586146 (cit. on p. 12).

Serag, Khaled et al. (2021). "Exposing New Vulnerabilities of Error Handling Mechanism in CAN". In: *30th USENIX Security Symposium* (cit. on p. 11).

Subaşı, Engin and Muharrem Mercimek (2026). "Real-Time, Reconfigurable CAN Intrusion Detection for EV Powertrain Networks via Specification-Driven Timing and Integrity Constraints". In: *Electronics* 15.9. DOI: 10.3390/electronics15091788 (cit. on pp. 6, 13).

Zhang, Haichun, Yan Meng, Xiaolin Yan, et al. (2020). "CANsec: A Practical In-Vehicle Controller Area Network Security Evaluation Tool". In: *Sensors* 20.17. DOI: 10.3390/s20174900 (cit. on p. 25).

Zhengyu, Han et al. (2019). "Research on Safe Communication Architecture for Real-Time Ethernet Distributed Control System". In: *IEEE Access* 7 (cit. on p. 5).